#3

Web Service

Spring & REST

*“ Tying up loose ends, finishing **Express Application Intro**: **Web Services** using **Spring Framework** (JAVA)”*

INNEHÅLLSFÖRTECKNING

01

ÖVERSIKT


Kursplan

Utbildningsmoment:

- Teoretiska och praktiska övningar
- Webbkommunikation
- Webbservices
- HTTP/HTTPS
- JSON
- XML
- Java
- Databaser
- Java applikationer


RESTful

RESTful?

Vad som är viktigt att inse är att REST, hur allmänt som helst, inte är en standard i sig, utan ett tillvägagångssätt, en stil, en uppsättning begränsningar för din arkitektur som kan hjälpa dig att bygga system i ‘web-scale’..

- Lämpliga åtgärder (GET, POST, PUT, DELETE, ...)
- Cachning
- Omdirigering och vidarebefordran
- Säkerhet (kryptering och autentisering)

02

Node.js & Database
Finalizing


How To
Query Documents

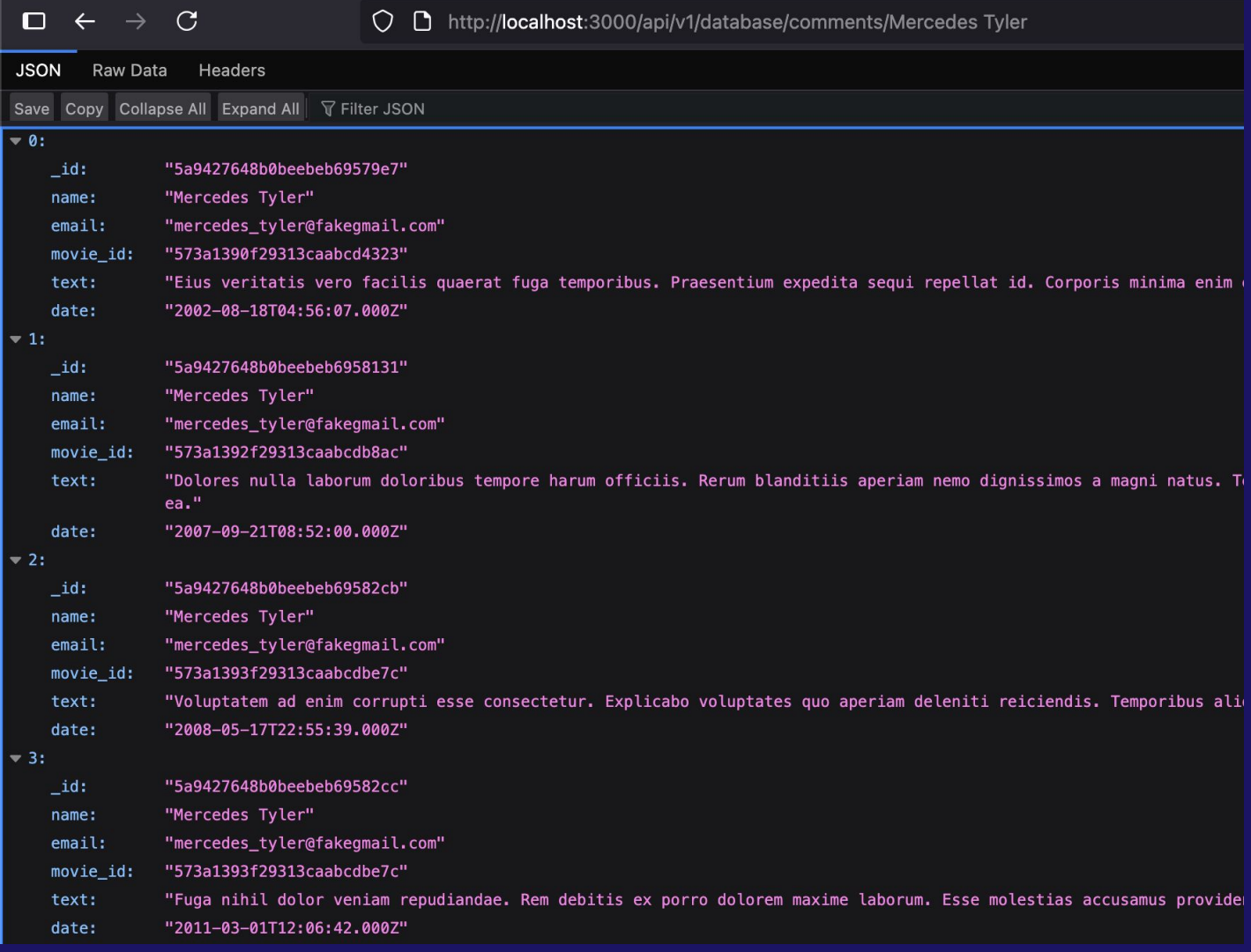
How to: Query Documents!
(Step #1)

```
app.get("/api/v1/database/comments/:username", (req, res) => {
 const db: Db = getDB()
 const result = db
   .collection("comments")
   .find({ name: req.params.username }) // Mercedes Tyler
   .limit(25)
   .toArray()

 res.send(result)
})
```

`.collection` - **hämtar samlingar** *(likt tabeller inom SQL)*
`.find()` - **letar efter nyckelord**
`.limit()` - **Begränsar antalet dokument som skickas vidare som resultat**

Error handling
(Step #2)

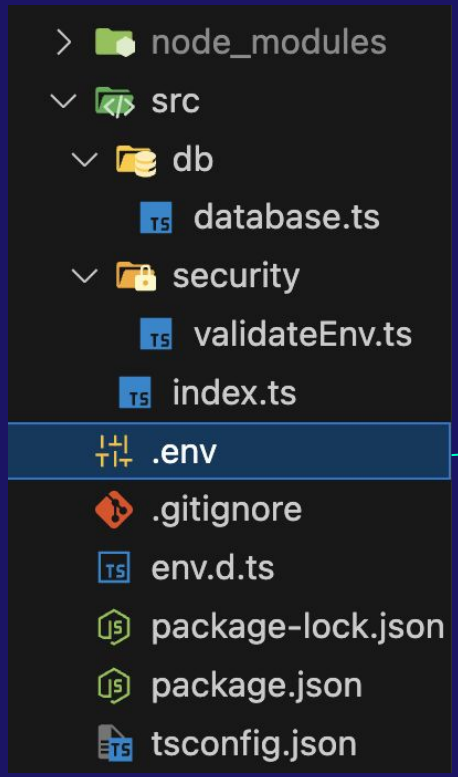
```
app.get("/api/v1/database/comments/:username", async (req, res) => {
 const db: Db = getDB()
 const result = await db
   .collection("comments")
   .find({ name: req.params.username }) // Mercedes Tyler
   .limit(25)
   .toArray()

 if (result.length === 0) {
   res.status(404).send({ message: "Nothing was found" })
   return
 }

 res.send(result)
})
```


http://localhost:3000/api/v1/database/comments/Mercedes Tyler
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
0:
_id: "5a9427648b0beebeb69579e7"
name: "Mercedes Tyler"
email: "mercedes_tyler@fakegmail.com"
movie_id: "573a1390f29313caabcd4323"
text: "Eius veritatis vero facilis quaerat fuga temporibus. Praesentium expedita sequi repellat id. Corporis minima enim
date: "2002-08-18T04:56:07.000Z"
1:
_id: "5a9427648b0beebeb6958131"
name: "Mercedes Tyler"
email: "mercedes_tyler@fakegmail.com"
movie_id: "573a1392f29313caabcdb8ac"
text: "Dolores nulla laborum doloribus tempore harum officiis. Rerum blanditiis aperiam nemo dignissimos a magni natus. T
ea."
date: "2007-09-21T08:52:00.000Z"
2:
_id: "5a9427648b0beebeb69582cb"
name: "Mercedes Tyler"
email: "mercedes_tyler@fakegmail.com"
movie_id: "573a1393f29313caabcdbe7c"
text: "Voluptatem ad enim corrupti esse consectetur. Explicabo voluptates quo aperiam deleniti reiciendis. Temporibus ali
date: "2008-05-17T22:55:39.000Z"
3:
_id: "5a9427648b0beebeb69582cc"
name: "Mercedes Tyler"
email: "mercedes_tyler@fakegmail.com"
movie_id: "573a1393f29313caabcdbe7c"
text: "Fuga nihil dolor veniam repudiandae. Rem debitis ex porro dolorem maxime laborum. Esse molestias accusamus provide
date: "2011-03-01T12:06:42.000Z"

RESULTAT

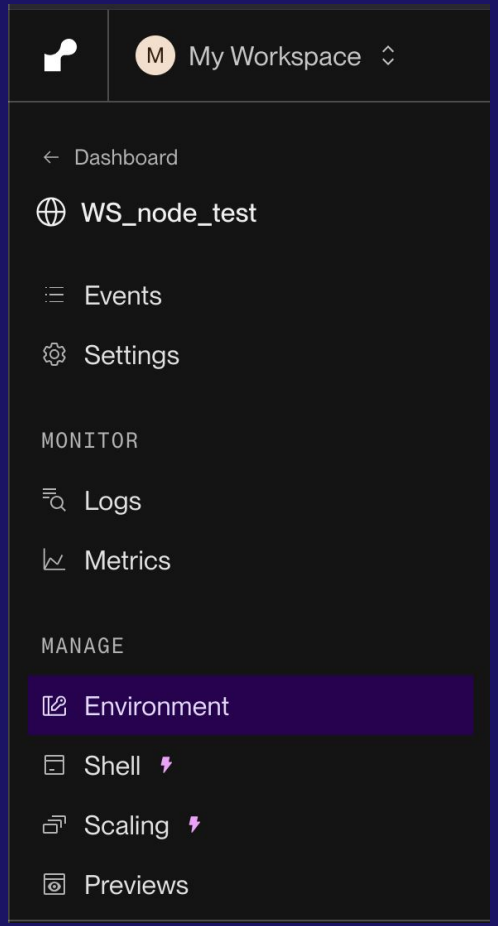

How to
Host using Render

The problem
(Explained)



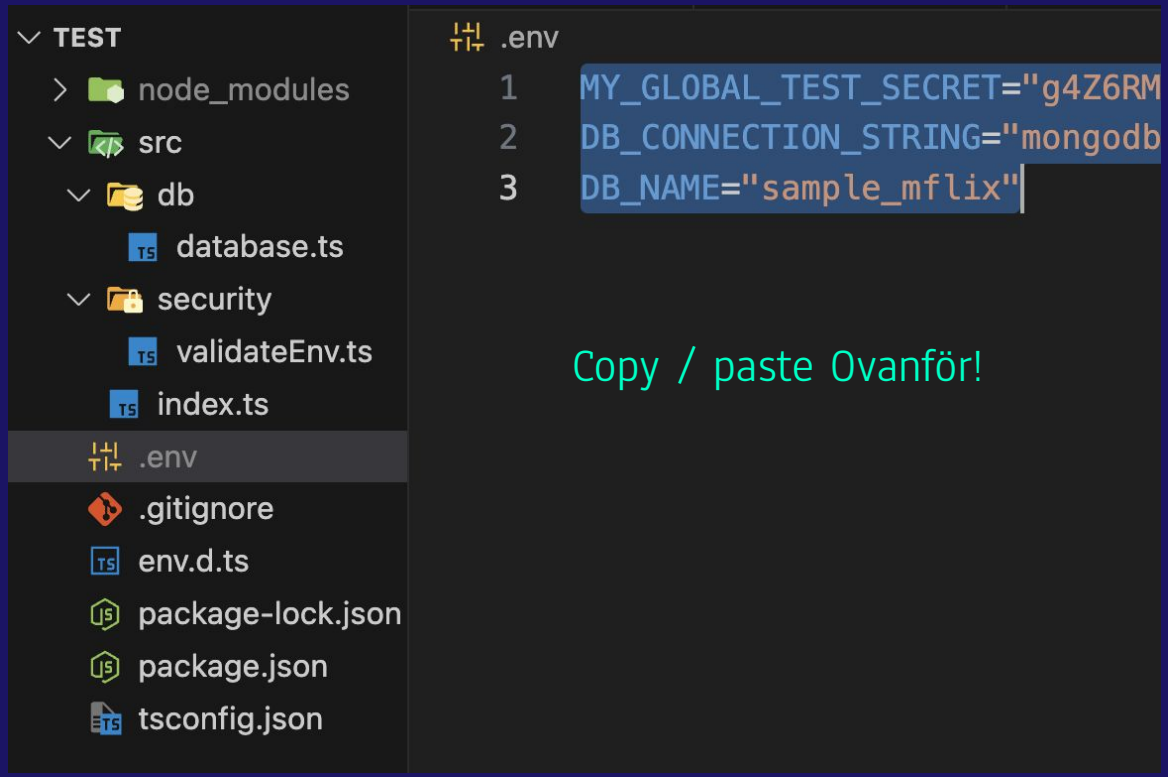
Enbart Lokalt...

Solution
(Part #1 - Login)

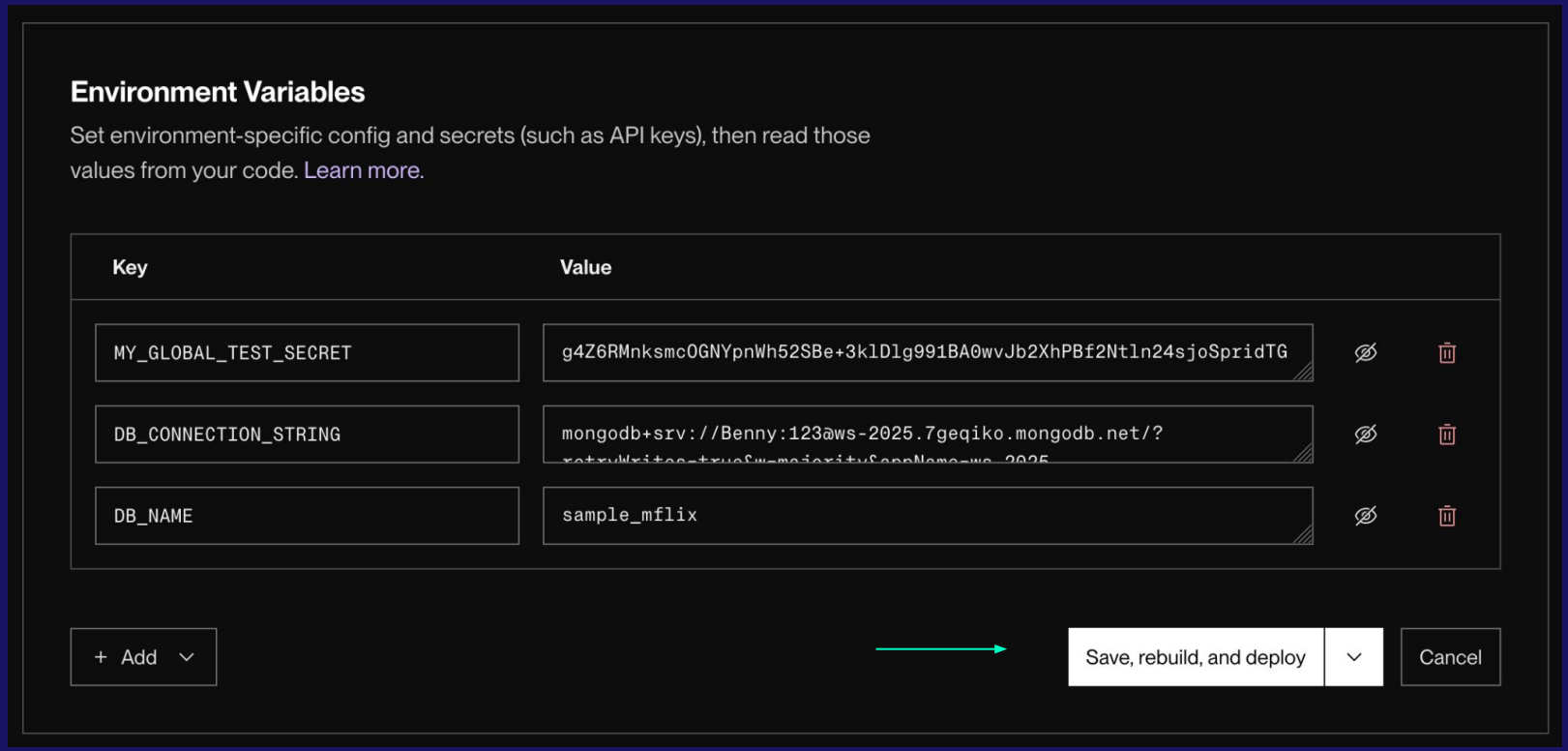


Navigera hit

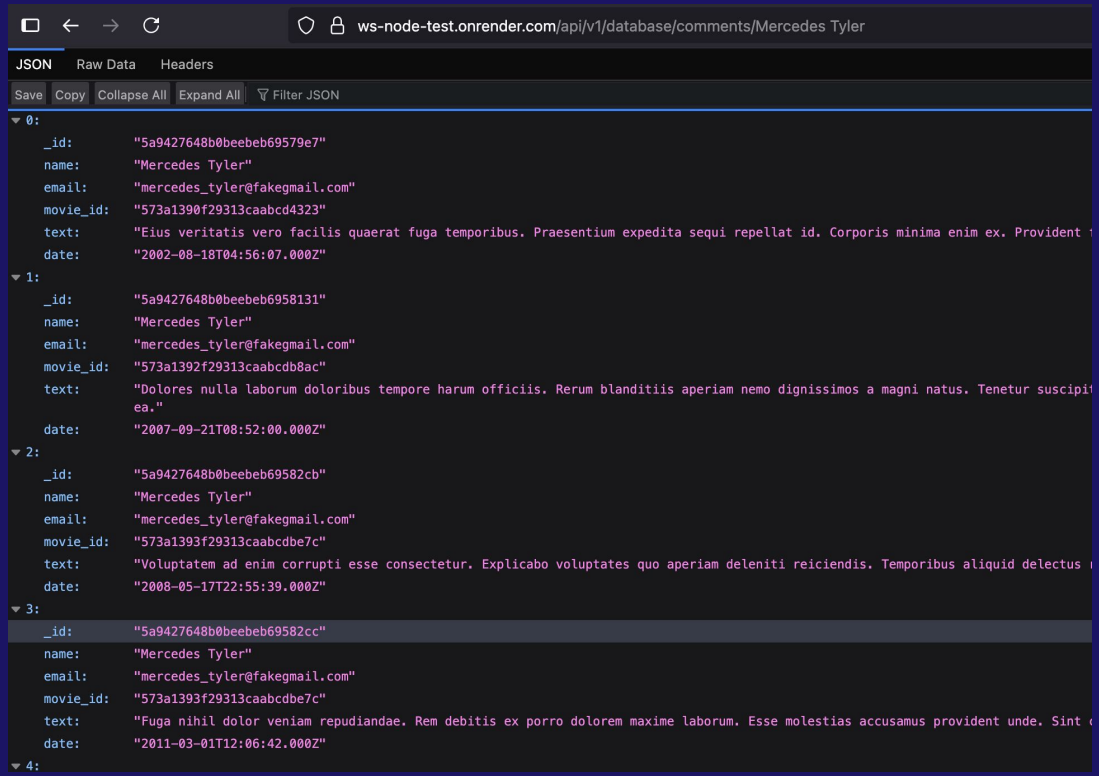
Solution
(Part #2 - Copy / Paste .env content)



Solution
(Part #3 – Results)

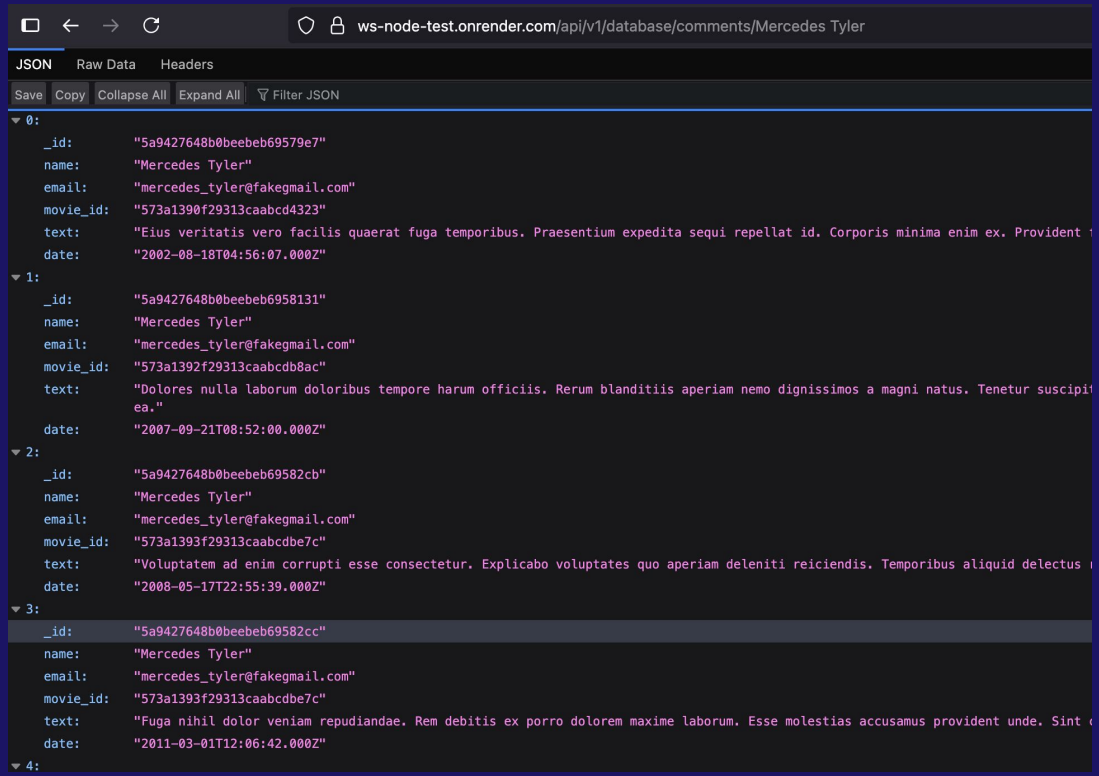


Results
(Part #4 - Production)



Notera - vi är nu LIVE

Results
(Part #4 - Production)



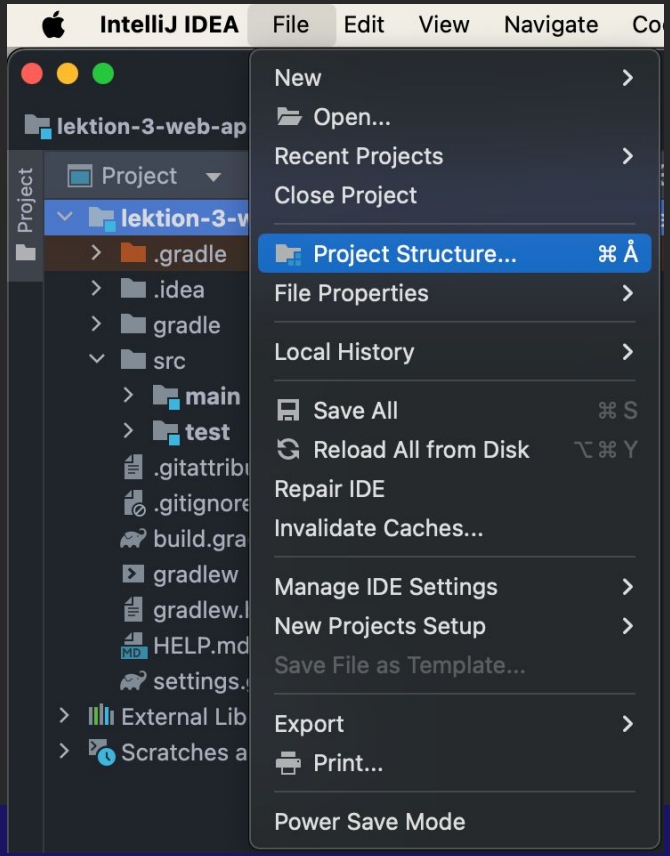
Notera - vi är nu LIVE

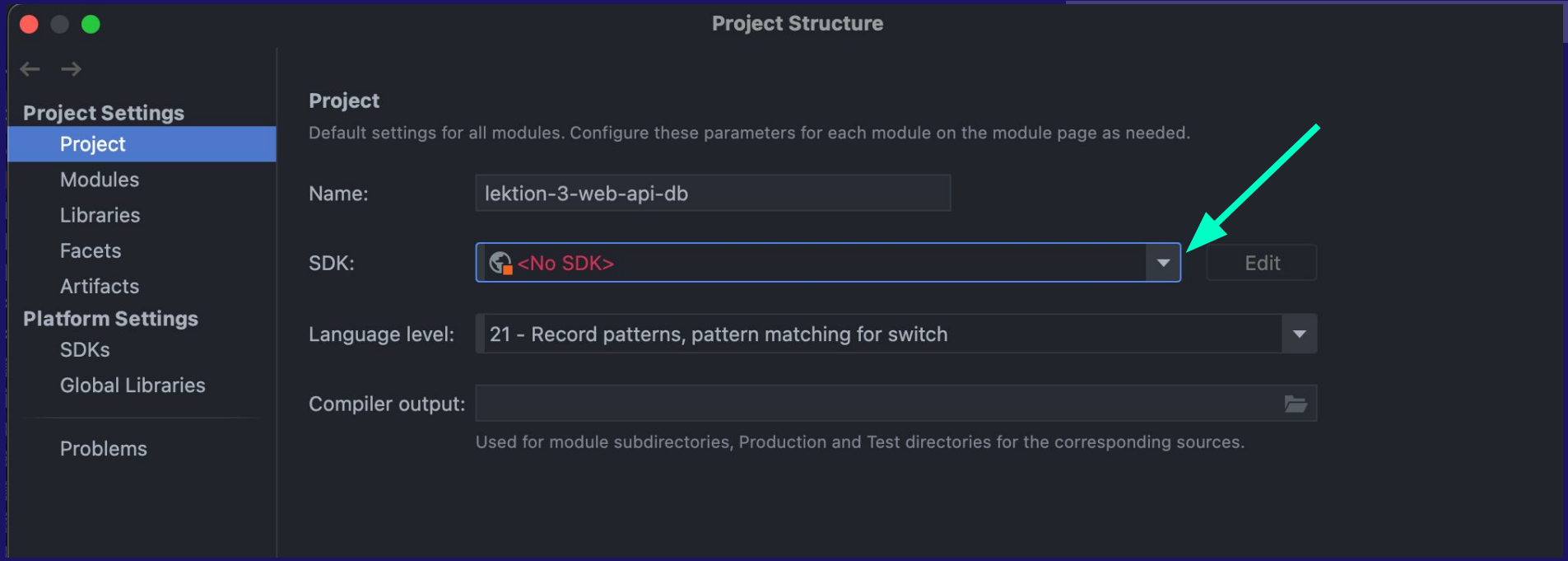
03

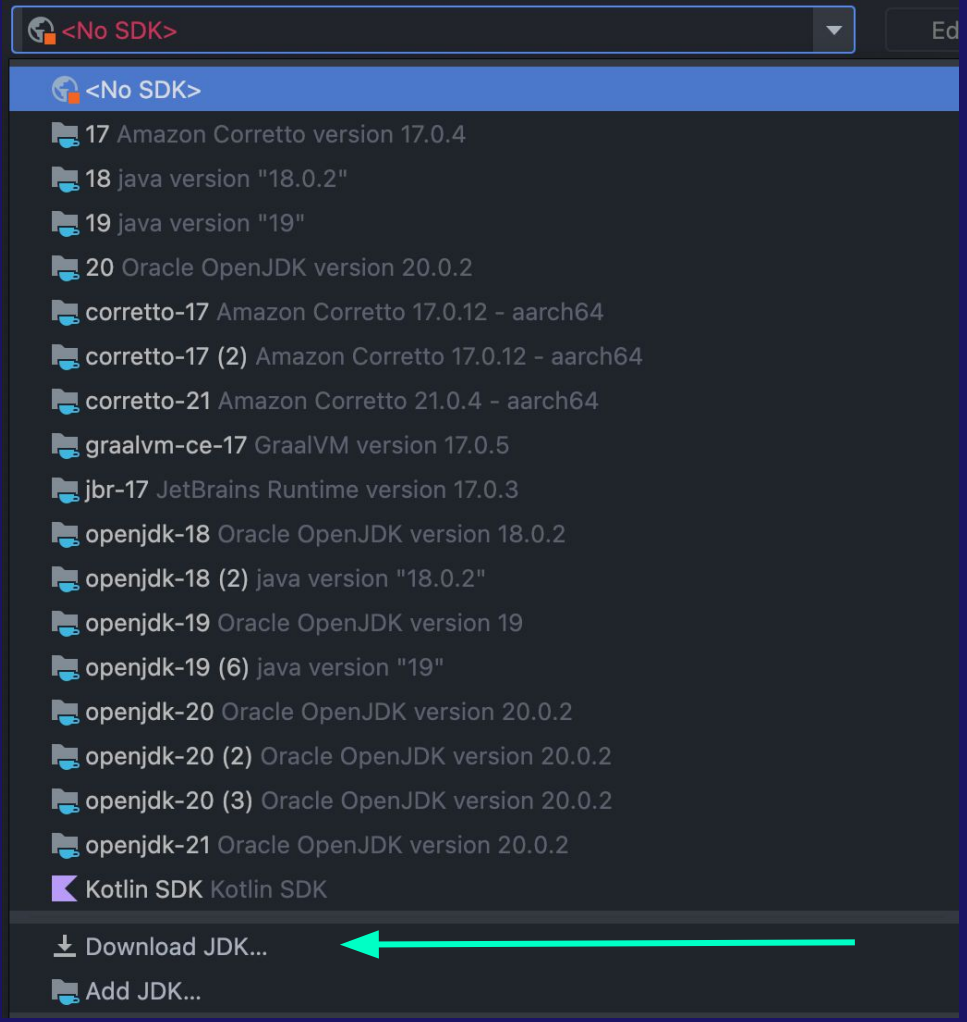
Restful & Spring Example


Säkerställ
Version
GRADLE & JDK/SDK

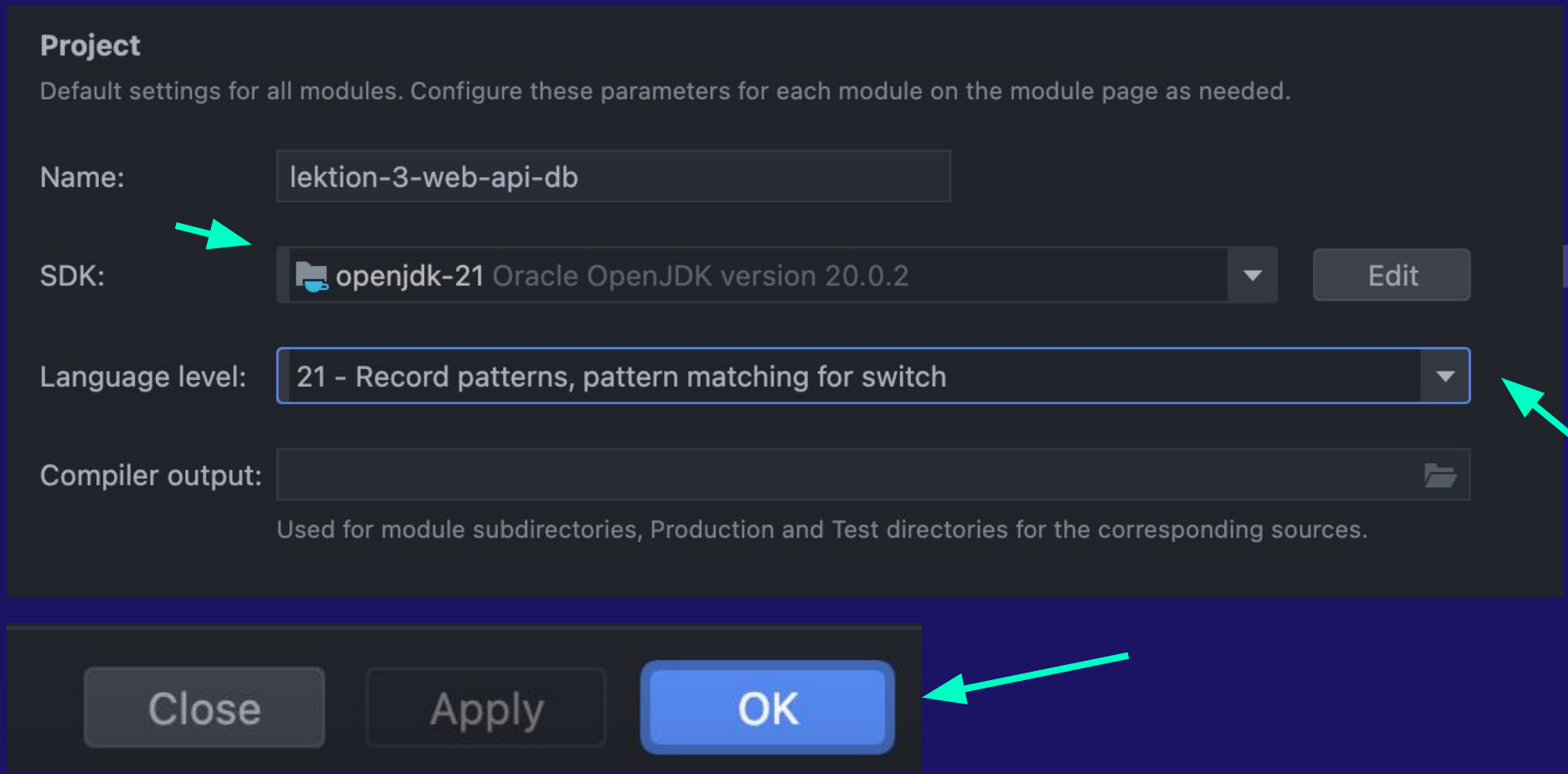
Navigera till Project Structure (Kan se annorlunda ut på Windows)

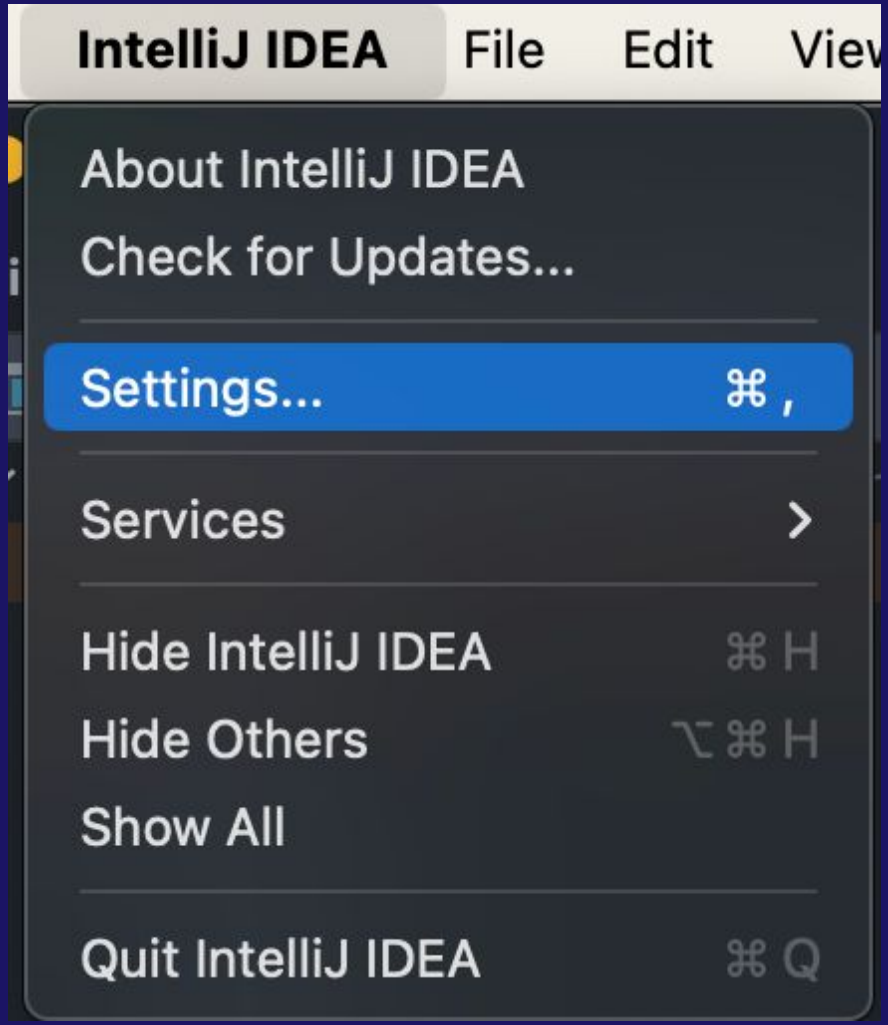


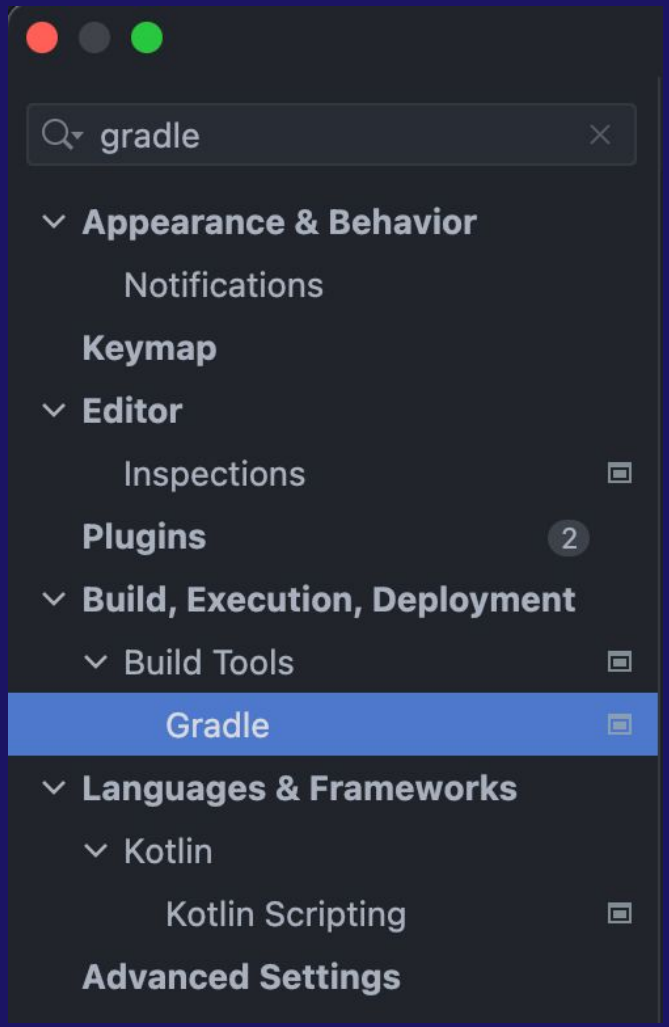

Project Structure
Project Settings
Project
Modules
Libraries
Facets
Artifacts
Platform Settings
SDKs
Global Libraries
Problems
Project
Default settings for all modules. Configure these parameters for each module on the module page as needed.
Name:
lektion-3-web-api-db
SDK:
<No SDK>
Edit
Language level:
21 - Record patterns, pattern matching for switch
Compiler output:
Used for module subdirectories, Production and Test directories for the corresponding sources.

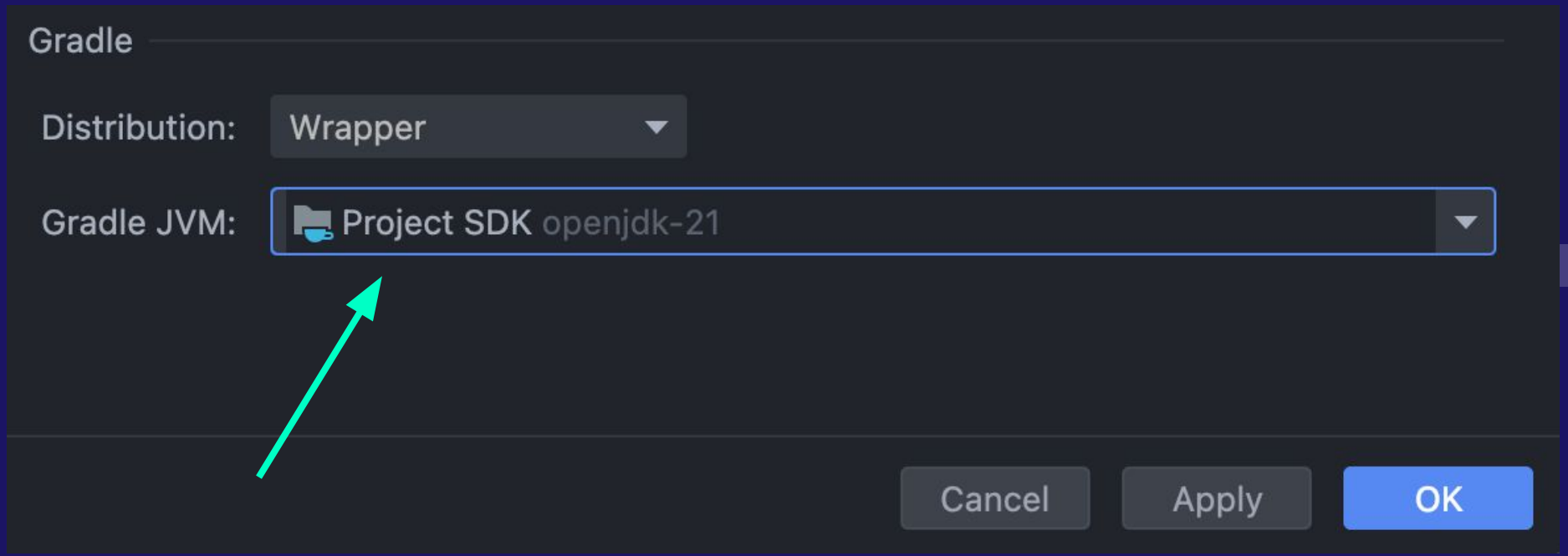

<No SDK>
Ed
<No SDK>
17 Amazon Corretto version 17.0.4
18 java version "18.0.2"
19 java version "19"
20 Oracle OpenJDK version 20.0.2
corretto-17 Amazon Corretto 17.0.12 - aarch64
corretto-17 (2) Amazon Corretto 17.0.12 - aarch64
corretto-21 Amazon Corretto 21.0.4 - aarch64
graalvm-ce-17 GraalVM version 17.0.5
jbr-17 JetBrains Runtime version 17.0.3
openjdk-18 Oracle OpenJDK version 18.0.2
openjdk-18 (2) java version "18.0.2"
openjdk-19 Oracle OpenJDK version 19
openjdk-19 (6) java version "19"
openjdk-20 Oracle OpenJDK version 20.0.2
openjdk-20 (2) Oracle OpenJDK version 20.0.2
openjdk-20 (3) Oracle OpenJDK version 20.0.2
openjdk-21 Oracle OpenJDK version 20.0.2
Kotlin SDK Kotlin SDK
Download JDK...
Add JDK...


Download JDK
Version:
21
Vendor:
Amazon Corretto 21.0.5 aarch64
Location:
~/Library/Java/JavaVirtualMachines/corretto-21.0.5
Cancel
Download


Project
Default settings for all modules. Configure these parameters for each module on the module page as needed.
Name:
lektion-3-web-api-db
SDK:
openjdk-21 Oracle OpenJDK version 20.0.2
Edit
Language level:
21 - Record patterns, pattern matching for switch
Compiler output:
Used for module subdirectories, Production and Test directories for the corresponding sources.
Close
Apply
OK


IntelliJ IDEA
File
Edit
View
About IntelliJ IDEA
Check for Updates...
Settings... ⌘ ,
Services
Hide IntelliJ IDEA ⌘ H
Hide Others ⌥ ⌘ H
Show All
Quit IntelliJ IDEA ⌘ Q


gradle
Appearance & Behavior
Notifications
Keymap
Editor
Inspections
Plugins
2
Build, Execution, Deployment
Build Tools
Gradle
Languages & Frameworks
Kotlin
Kotlin Scripting
Advanced Settings


Gradle
Distribution:
Wrapper
Gradle JVM:
Project SDK openjdk-21
Cancel
Apply
OK


Status Codes

Överblick

Sätt att kommunicera på,
via Web Services.

https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml#http-status-codes-1

418

The HTTP status code 418 is a playful and non-standard code known as "I'm a teapot."

It was defined in RFC 2324, which is an April Fools' joke specification called the **Hyper Text Coffee Pot Control Protocol (HTCPCP/1.0)**.

The protocol was created as a parody, designed to control, monitor, and diagnose coffee pots over the internet


Annotationer
AKA
@

Annotation

Vad är en annotation?

@Override

Annotation

Vad är en annotation?

Syntaktisk metadata...?

@Override?

https://www.geeksforgeeks.org/annotations-in-java/


Spring

Definitioner

Security!

Easy setup!

Enterprise!

Annotations!

Modern!

API / WS!

Spring

Spring Ramverk,
tillhandahåller en omfattande
programmerings- och konfigurationsmodell
för moderna Java-baserade företagsapplikationer

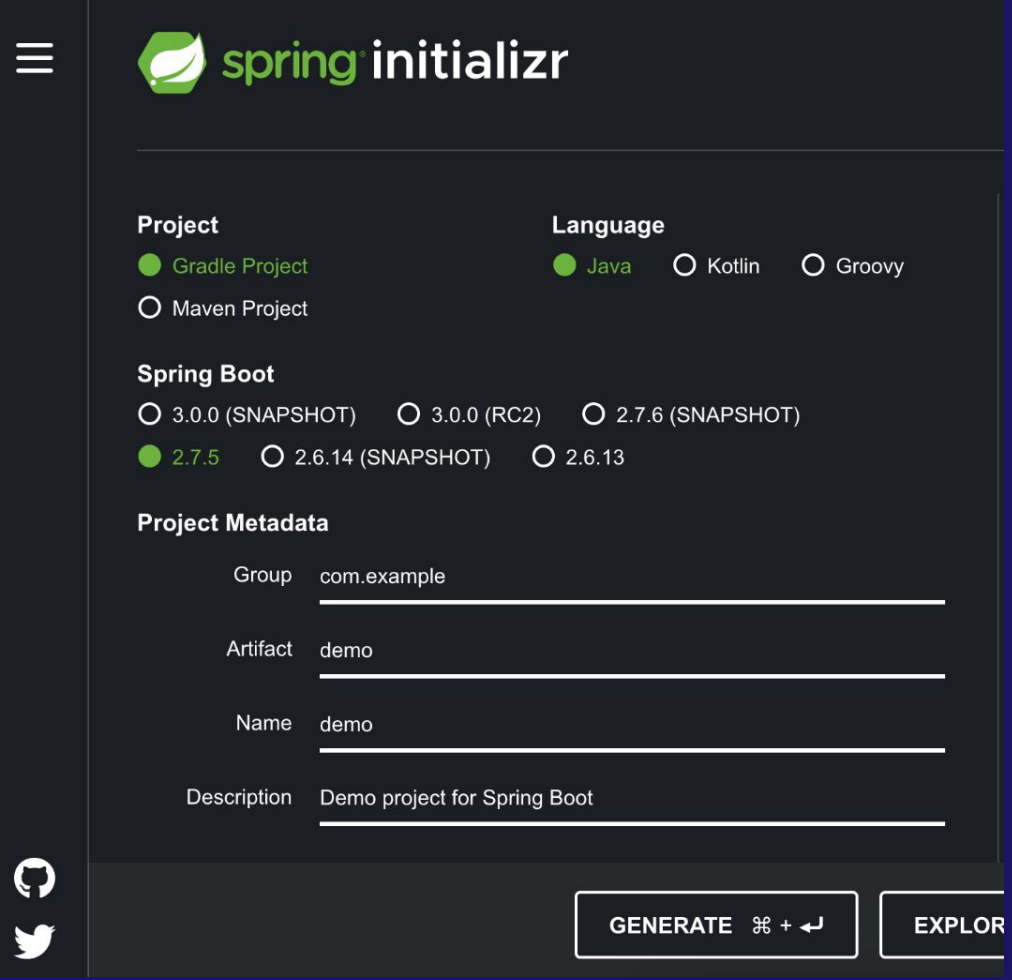
Definitioner

spring initializr

initializr

Skapar ett färdigt projekt som agerar som en ‘mall’

https://start.spring.io/



Project – Gradle VS Maven (Feel free to pick anything)

Language – Java (don't touch)

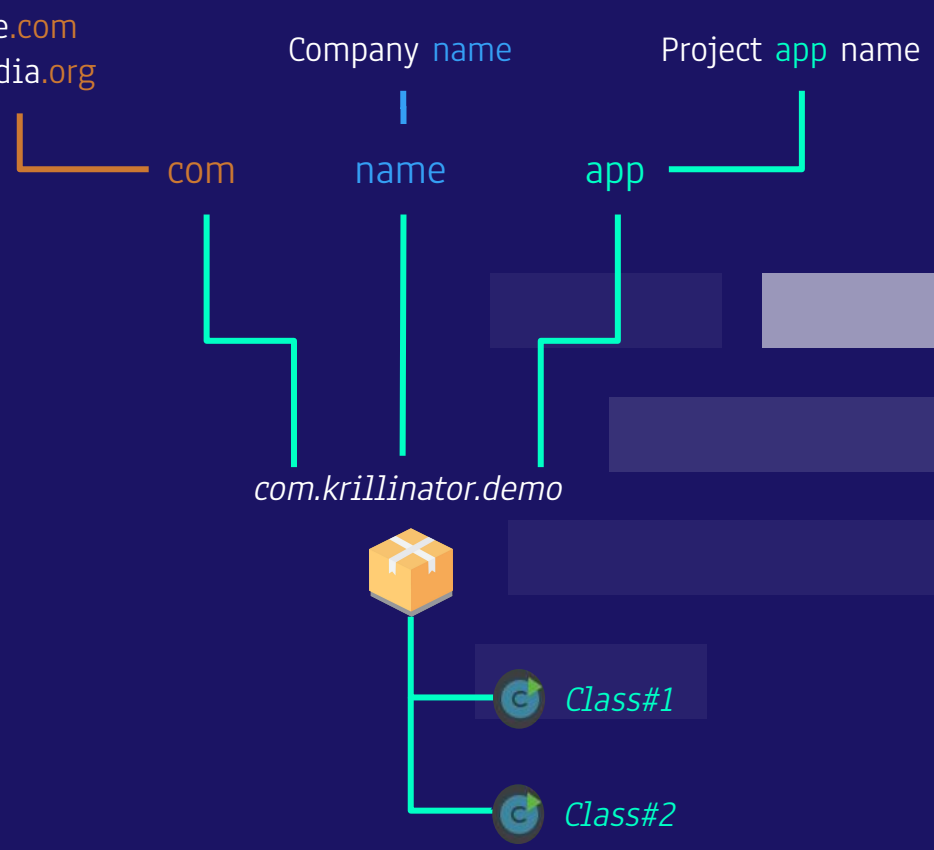
Spring Boot – Default (don't touch)

Project Metadata:

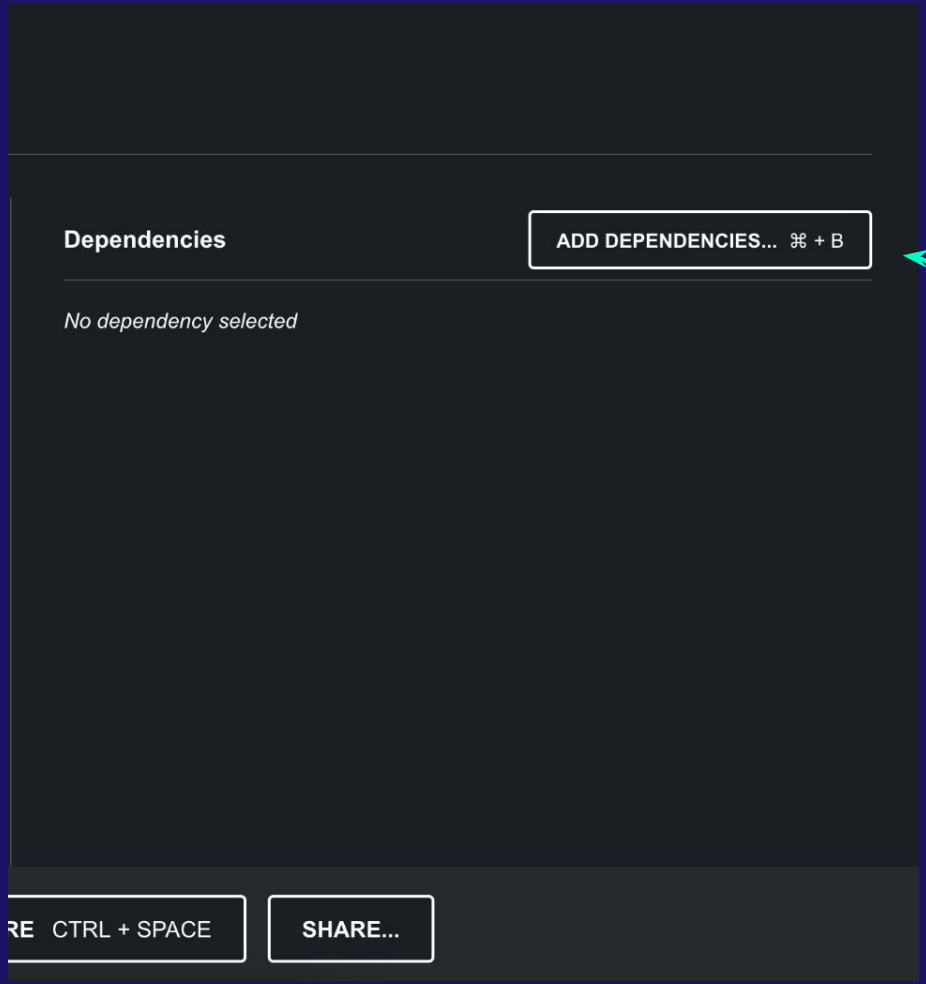
- Group - com.COMPANY_NAME (can edit)
- Artifact - PROJECT_NAME (can edit)
- Name - (no need to touch)
- Description - (no need to touch)

Packages

Med paket strukturerar vi upp projektet med olika filer såsom klasser!



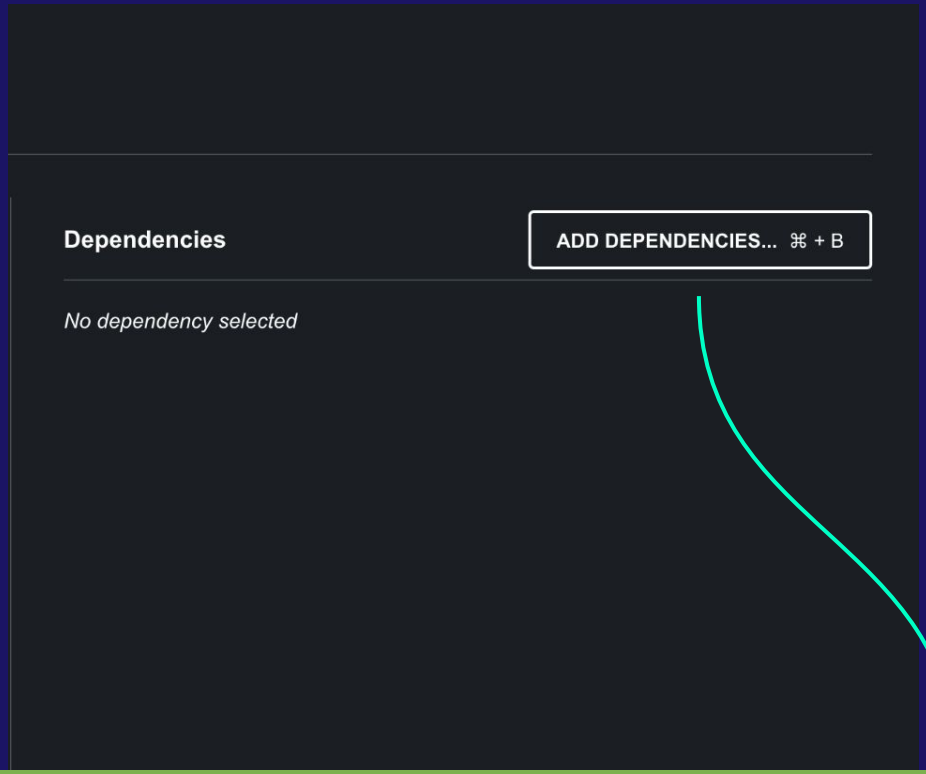
Alternativt kan man även skriva: *com.name.region.app*



Add Dependencies - Add any kind of dependency!

Everything from MYSQL, Postgres, MongoDB to dev tools, GUI with Thymeleaf to cloud, API & web sockets

(Utforska gärna!)



SPRING WEB (WEB)

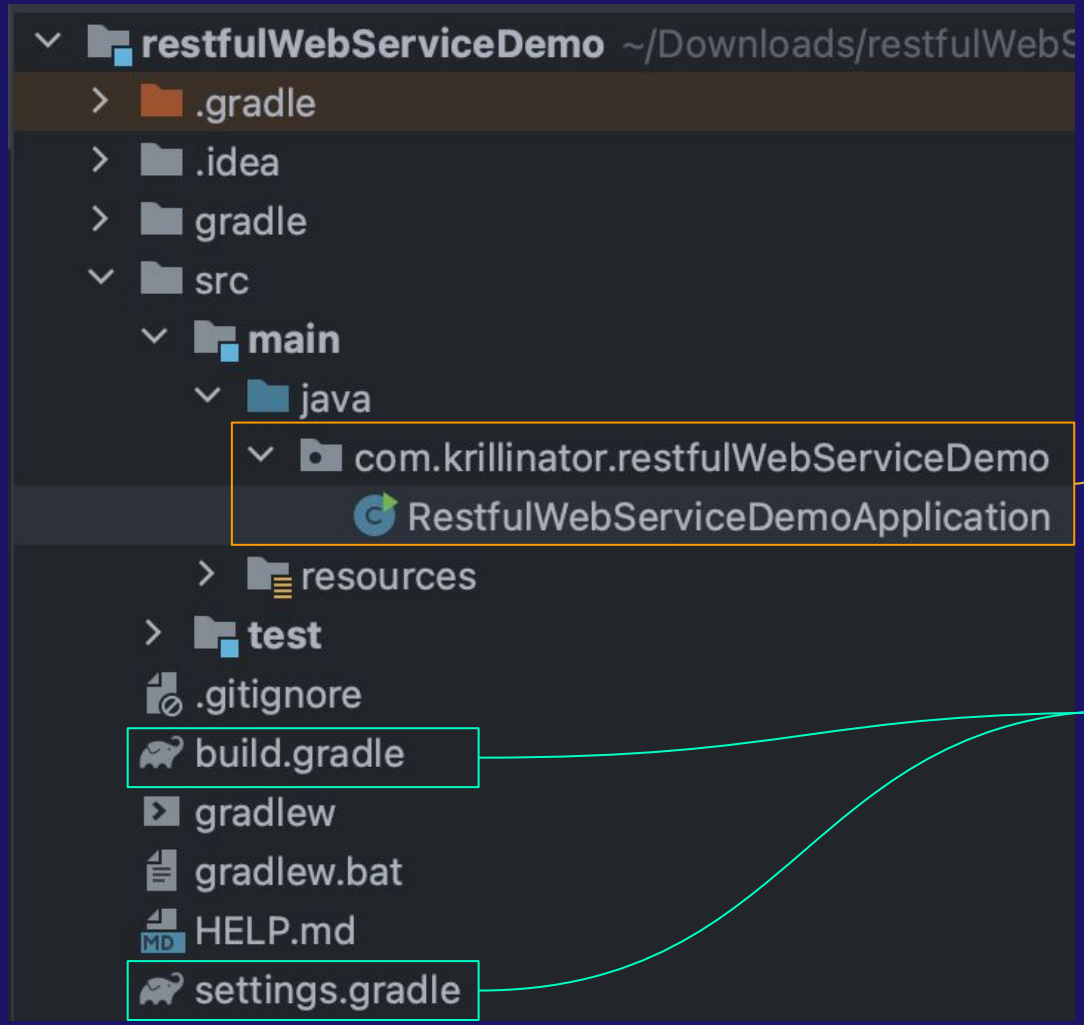
Apache Tomcat ‘Default Container’

Använder sig av Spring MVC

Inkluderar ‘RESTful’ + web support

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.



Project Structure

Strukturen på projektet ser lite annorlunda ut..

Vi har vår MAIN som nu heter:
RestfulWebServiceDemoApplication

Märk även av 'GRADLE'!

RestfulWebServiceDemoApplication.java

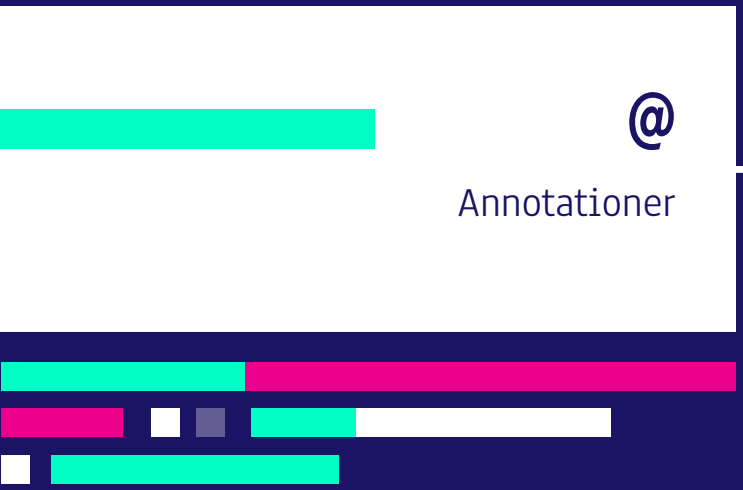
```java
package com.krillinator.restfulWebServiceDemo;

import ...

@SpringBootApplication
public class RestfulWebServiceDemoApplication {

    public static void main(String[] args) { SpringApplication.run(RestfulWebServiceDemoApplication.class, args); }

}
```

Vår första 'Annotation'

Resterande ser relativt 'normalt' ut. Låt oss kika lite närmre på Annotationen!


RESTful + Spring

ANNOTATIONS – RESTful



@SpringBootApplication

Startpunkten på vår app! Innehåller flertal annotationer under huven.

@RestController

Definierar en Web Service ‘controller’ (MVC struktur)

@GetMapping()

Definierar en endpoint som parameter där vi exekverar vår logik

@RequestParam()

‘Endpoint’ kan ta in en ‘optional’ parameter för dynamiska värden!

@SpringBootApplication

Information är viktigt, men låt oss bryta ner det stegvist

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = {@org.springframework.context.annotation.ComponentScan.Filter(type = org.springframework
public @interface SpringBootApplication
extends java.lang.annotation.Annotation
```

Indicates a `configuration` class that declares one or more `@Bean` methods and also triggers `auto-configuration` and `component scanning`. This is a convenience annotation that is equivalent to declaring `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`.

Since: 1.2.0

Author: Phillip Webb, Stephane Nicoll, Andy Wilkinson

Gradle: org.springframework.boot:spring-boot-autoconfigure:2.7.5 (spring-boot-autoconfigure-2.7.5.jar)

@SpringBootApplication

Deklarerar @Bean... Auto-config... Component Scanning...
Innehåller även
@Configuration,
@EnableAutoConfiguration &
@ComponentScan

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = {@org.springframework.context.annotation.ComponentScan.Filter(type = org.springframework
public @interface SpringBootApplication
extends java.lang.annotation.Annotation
```

Indicates a `configuration` class that declares one or more `@Bean` methods and also triggers `auto-configuration` and `component scanning`. This is a convenience annotation that is equivalent to declaring `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`.

Since: 1.2.0

Author: Phillip Webb, Stephane Nicoll, Andy Wilkinson

Gradle: org.springframework.boot:spring-boot-autoconfigure:2.7.5 (spring-boot-autoconfigure-2.7.5.jar)

@SpringBootApplication

Deklarerar @Bean... Auto-config... Component Scanning...
Innehåller även
@Configuration,
@EnableAutoConfiguration &
@ComponentScan

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(excludeFilters = {@org.springframework.context.annotation.ComponentScan.Filter(type = org.springframework
public @interface SpringBootApplication
extends java.lang.annotation.Annotation
```

Indicates a configuration class that declares one or more @Bean methods and also triggers auto-configuration and component scanning. This is a convenience annotation that is equivalent to declaring @Configuration, @EnableAutoConfiguration and @ComponentScan.

Since: 1.2.0

Author: Phillip Webb, Stephane Nicoll, Andy Wilkinson

Gradle: org.springframework.boot:spring-boot-autoconfigure:2.7.5 (spring-boot-autoconfigure-2.7.5.jar)

@SpringBootApplication

Varför går vi igenom dessa annotationer?

Det finns en risk att ni kommer stöta på märkliga problem som ni bara kan lösa om ni har förståelse över hur 'Spring' fungerar i grund..

Annotationer är alltså nyckeln till problemen!

Running the app

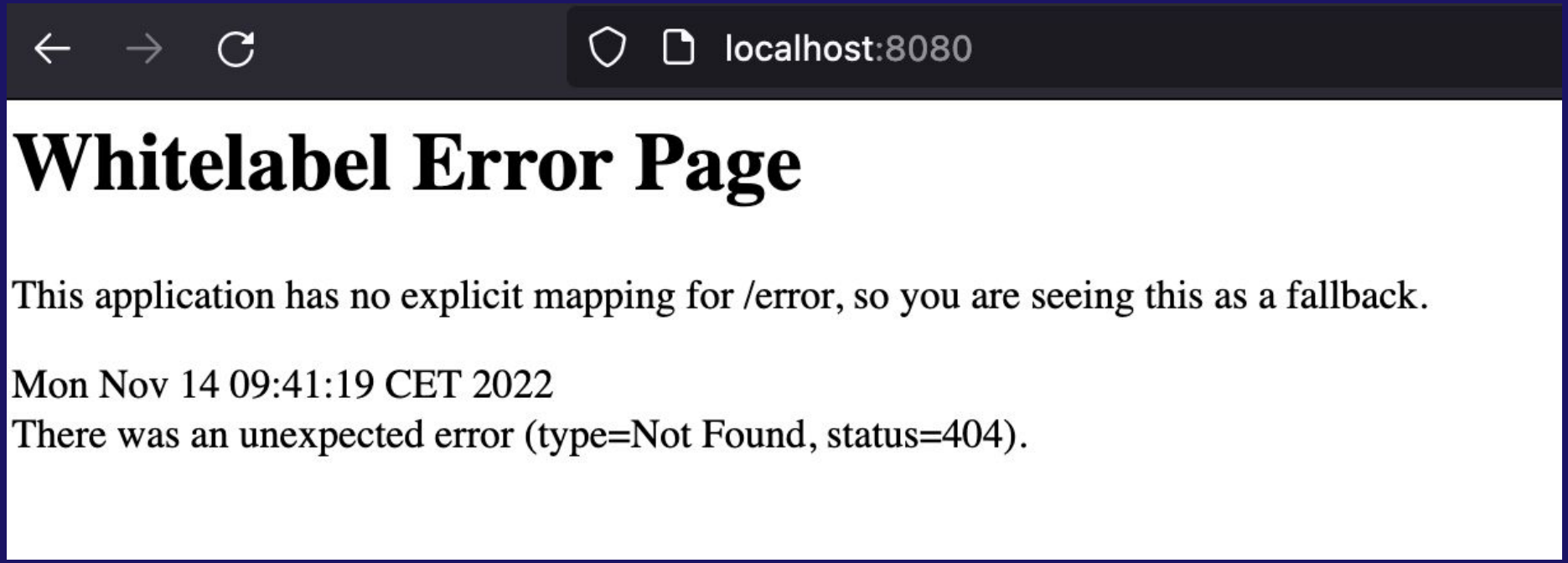
Starta appen!

“Tomcat initialized with port: 8080 (HTTP)” ← Prova att gå in på denna adress genom att skriva i din webbläsare:

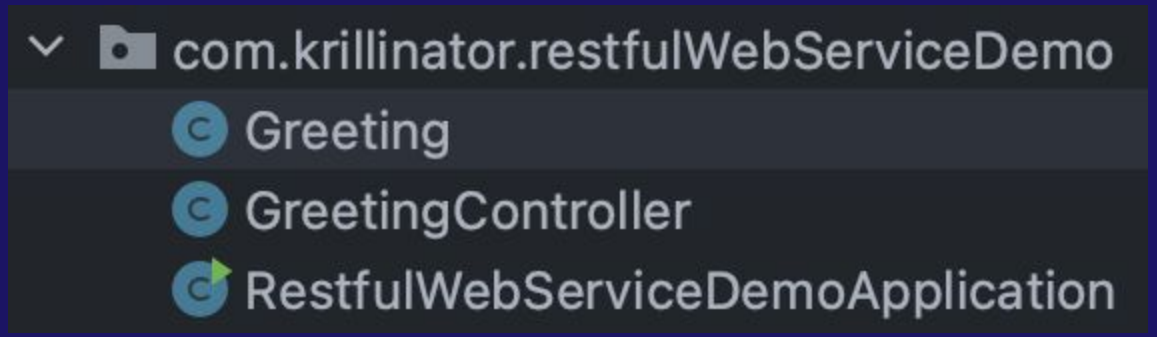
localhost:8080

```
2022-11-14 09:40:43.279  INFO 9894 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2022-11-14 09:40:43.286  INFO 9894 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2022-11-14 09:40:43.286  INFO 9894 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.68]
```

Vad har hänt!?

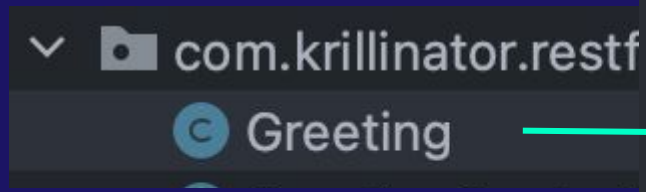

localhost:8080
Whitelabel Error Page
This application has no explicit mapping for /error, so you are seeing this as a fallback.
Mon Nov 14 09:41:19 CET 2022
There was an unexpected error (type=Not Found, status=404).


Att börja koda!
#1



Låt oss börja koda lite...

Vi börjar med att skapa två klasser: Greeting & GreetingController



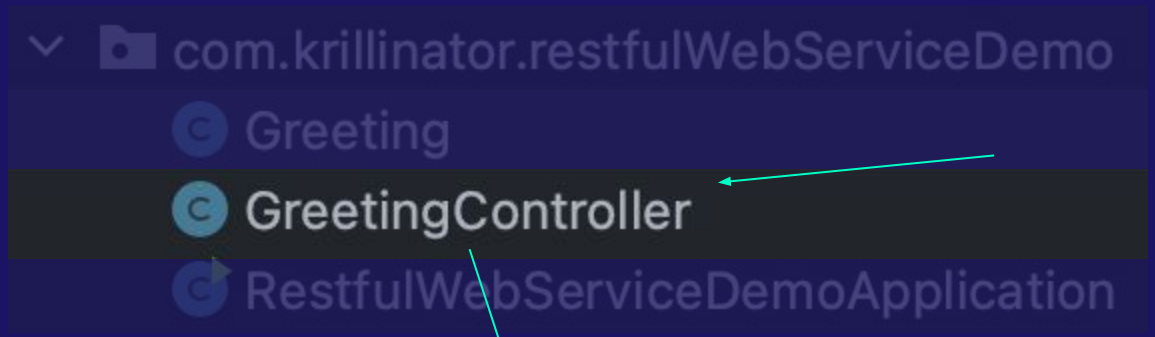
Private final??

Long?

Enbart ‘getters’?

Constructor…?

```java
package com.krillinator.restfulWebServiceDemo;

public class Greeting {

    private final long id;
    private final String content;

    public Greeting(long id, String content) {
        this.id = id;
        this.content = content;
    }

    public long getId() {
        return id;
    }

    public String getContent() {
        return content;
    }
}
```




```
9   @RestController
10  public class GreetingController {
```

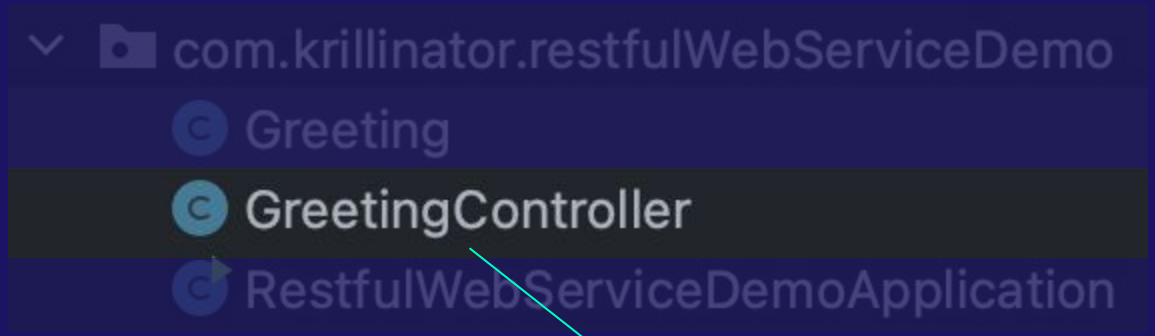
Definitioner

@RestController

@RestController

Inkluderar @ResponseBody inuti sig!
Mindre syntax för Web Services!

https://www.baeldung.com/spring-controller-vs-restcontroller



@GetMapping()

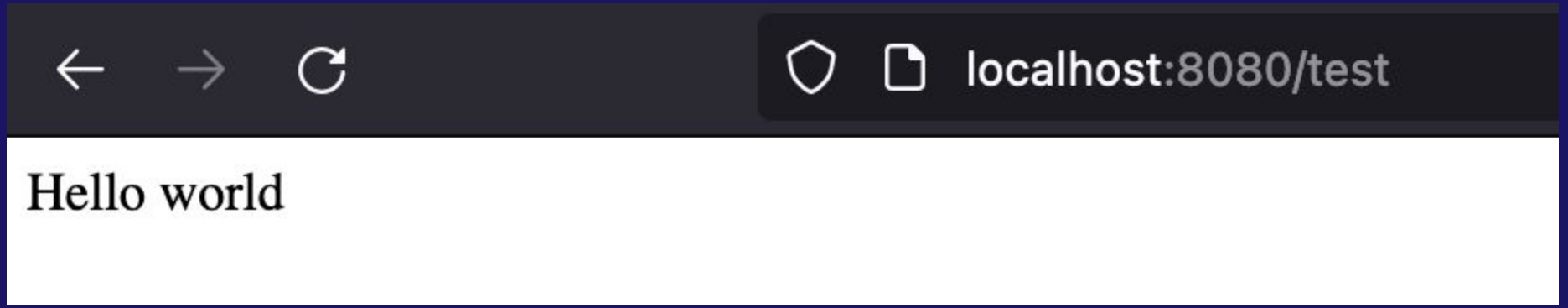
En anotation som tar in en ‘Endpoint’

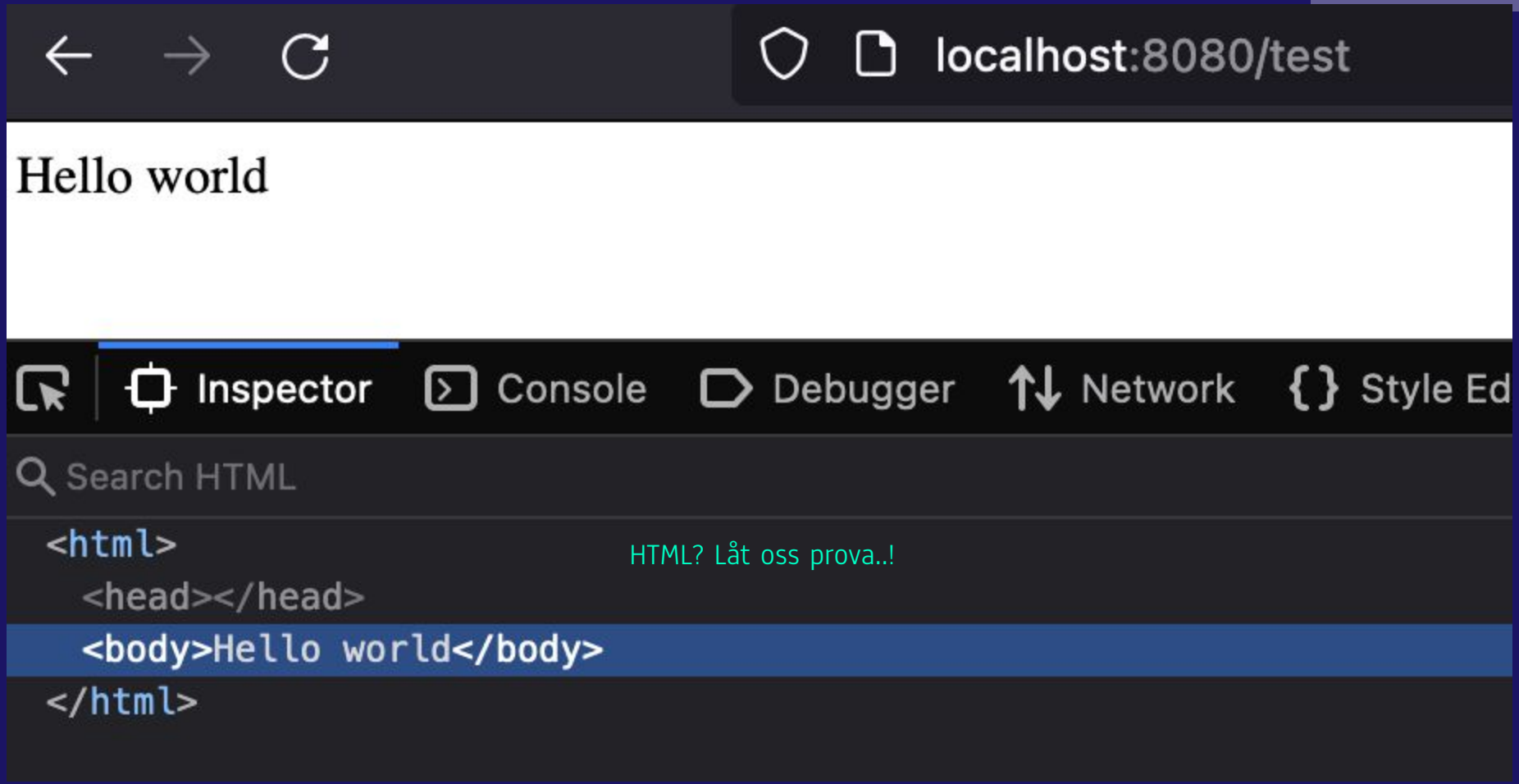
Vi returnerar en “Sträng”
Men vad returneras egentligen när vi går in på sidan…?

```java
@GetMapping("/test")
public String test() {

    return "Hello world";
}
```

Ok vi ser att den returnerar “Hello world”... men vad exakt är detta?

Är det JSON..?



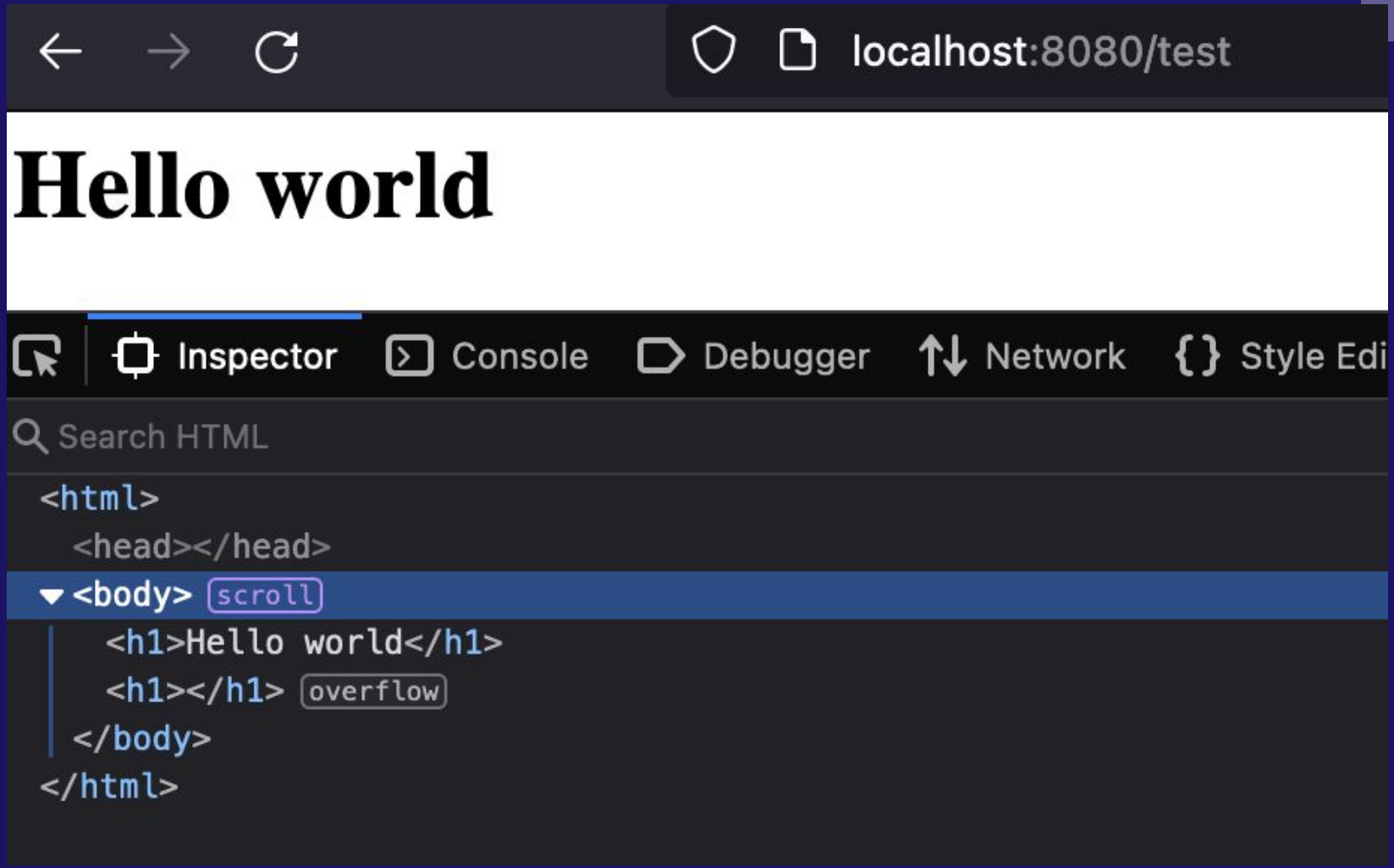

localhost:8080/test
Hello world
Inspector
Console
Debugger
Network
Style Ed
Search HTML
<html>
<head></head>
<body>Hello world</body>
</html>
HTML? Låt oss prova..!

Before...

```
    return "Hello world";
```

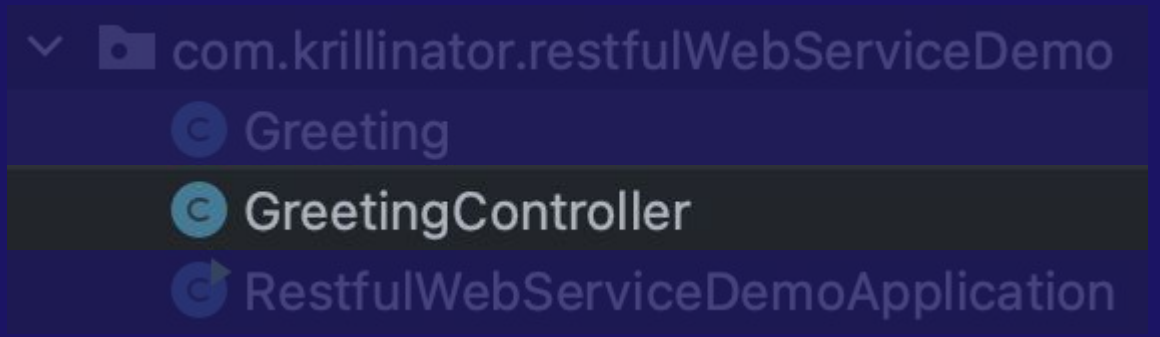
After!

```
@GetMapping("/test")
public String test() {

    return "<h1>Hello world <h1/>";
}
```


localhost:8080/test
Hello world
Inspector
Console
Debugger
Network
Style Edi
Search HTML
<html>
<head></head>
<body> scroll
<h1>Hello world</h1>
<h1></h1> overflow
</body>
</html>


Att börja koda!
#2

Låt oss gå igenom ytterligare ett exempel med ‘endpoints’!



```java
private static final String template = "Hello, %s";
```

Private *access modifier*	-	Åtkomst, enbart inom vår denna klass
'Static'	-	Global tillgång
'Final'	-	Är ej föränderlig
String	-	Datatyp
"%s"	-	Dynamisk Variable (likt SOUF)

```java
private static final String template = "Hello, %s";
private final AtomicLong counter = new AtomicLong();
```

AtomicLong - Datatyp / Klass, med inbyggda metoder (Debugging), use when working with threads

new - new

```
private static final String template = "Hello, %s";
private final AtomicLong counter = new AtomicLong();

@GetMapping("/greeting")
```

GetMapping() - Endpoint (t.ex. localhost:8080/greeting)

```java
private static final String template = "Hello, %s";
private final AtomicLong counter = new AtomicLong();

@GetMapping("/greeting")
public Greeting greeting(@RequestParam(value = "name", defaultValue = "World") String name ) {
```

Greeting - Retur typ

greeting - Metodnamn

@RequestParam() - Tar in ‘value’ och optional ‘default’

Source: https://www.baeldung.com/spring-request-param

```java
private static final String template = "Hello, %s";
private final AtomicLong counter = new AtomicLong();

@GetMapping("/greeting")
public Greeting greeting(@RequestParam(value = "name", defaultValue = "World") String name ) {
```

@RequestParam()

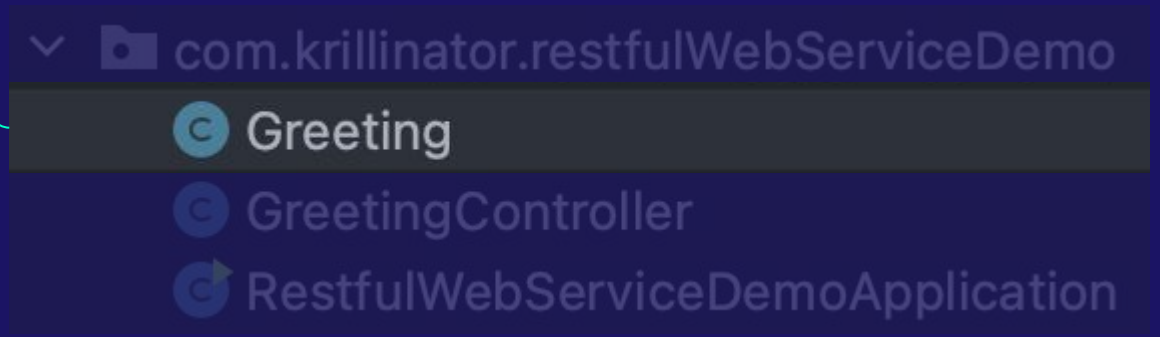
Value = ‘värdet vi matar in

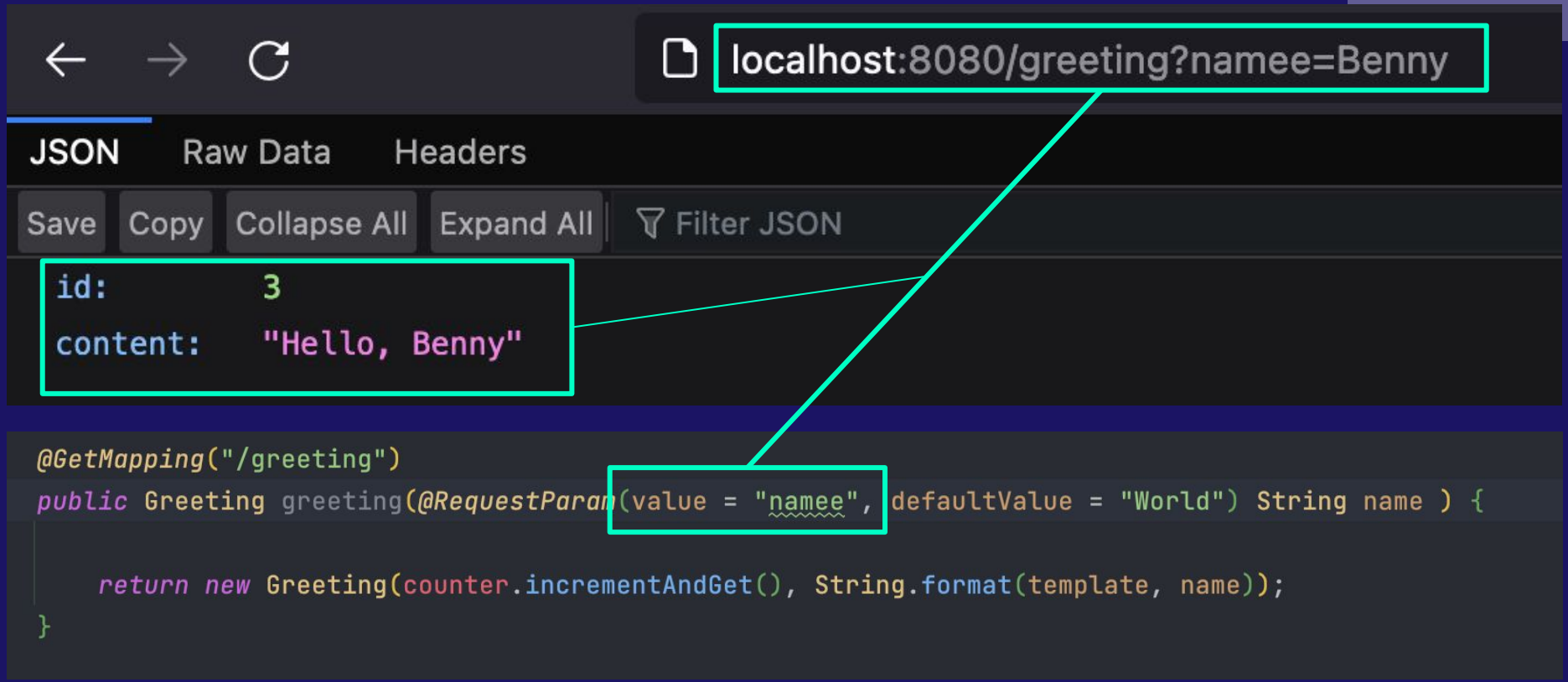
‘Värdet en referens variabel till värdet vi matat in

defaultValue = optional value om man råkat missa inmatning

```java
private static final String template = "Hello, %s";
private final AtomicLong counter = new AtomicLong();

@GetMapping("/greeting")
public Greeting greeting(@RequestParam(value = "name", defaultValue = "World") String name ) {

    return new Greeting(counter.incrementAndGet(), String.format(template, name));
}
```

Här returnerar vi en ny ‘klass’ som heter ‘Greeting()

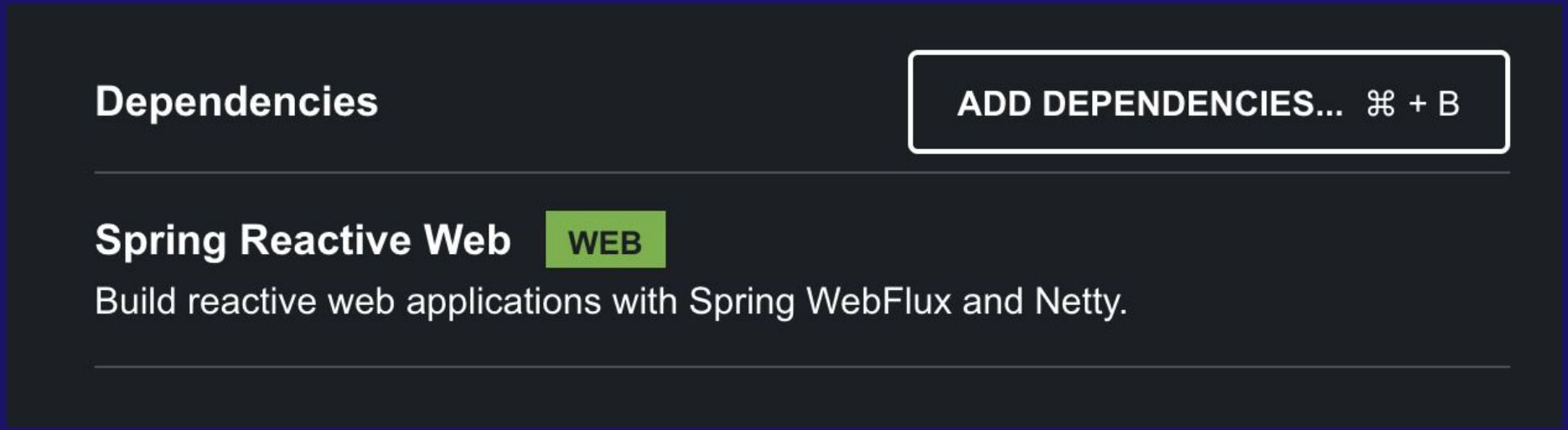




Se här att ‘value’ blir nyckelordet vi måste mata in! Annars fungerar det ej!


Async & Webflux

Flux
(Dependency)



FLUX
(Pitfall)

NOTE

Adding both `spring-boot-starter-web` and `spring-boot-starter-webflux` modules in your application results in Spring Boot auto-configuring Spring MVC, not WebFlux. This behavior has been chosen because many Spring developers add `spring-boot-starter-webflux` to their Spring MVC application to use the reactive `WebClient`. You can still enforce your choice by setting the chosen application type to `SpringApplication.setWebApplicationType(WebApplicationType.REACTIVE)`.

Definitioner

Webflux

Reactive Web (async)
Tillåter oss att arbeta asynkront!

Mono för singulär data
Flux för ström av data

https://docs.spring.io/spring-boot/reference/web/reactive.html

FLUX
(Example)

```
package com.example.demo.controller ;

import org.springframework.web.bind.annotation. GetMapping;
import org.springframework.web.bind.annotation. RestController;
import reactor.core.publisher.Mono ;

@RestController
public class GreetingController  {

    @GetMapping("/greeting")
    public Mono<String> sayHello () {

        return Mono.just("Hello World");
    }

}
```

Notera returtypen: ‘Mono’

Ersätt ‘Mono’ med ‘flux’ om ström av data önskas!


Kotlin?

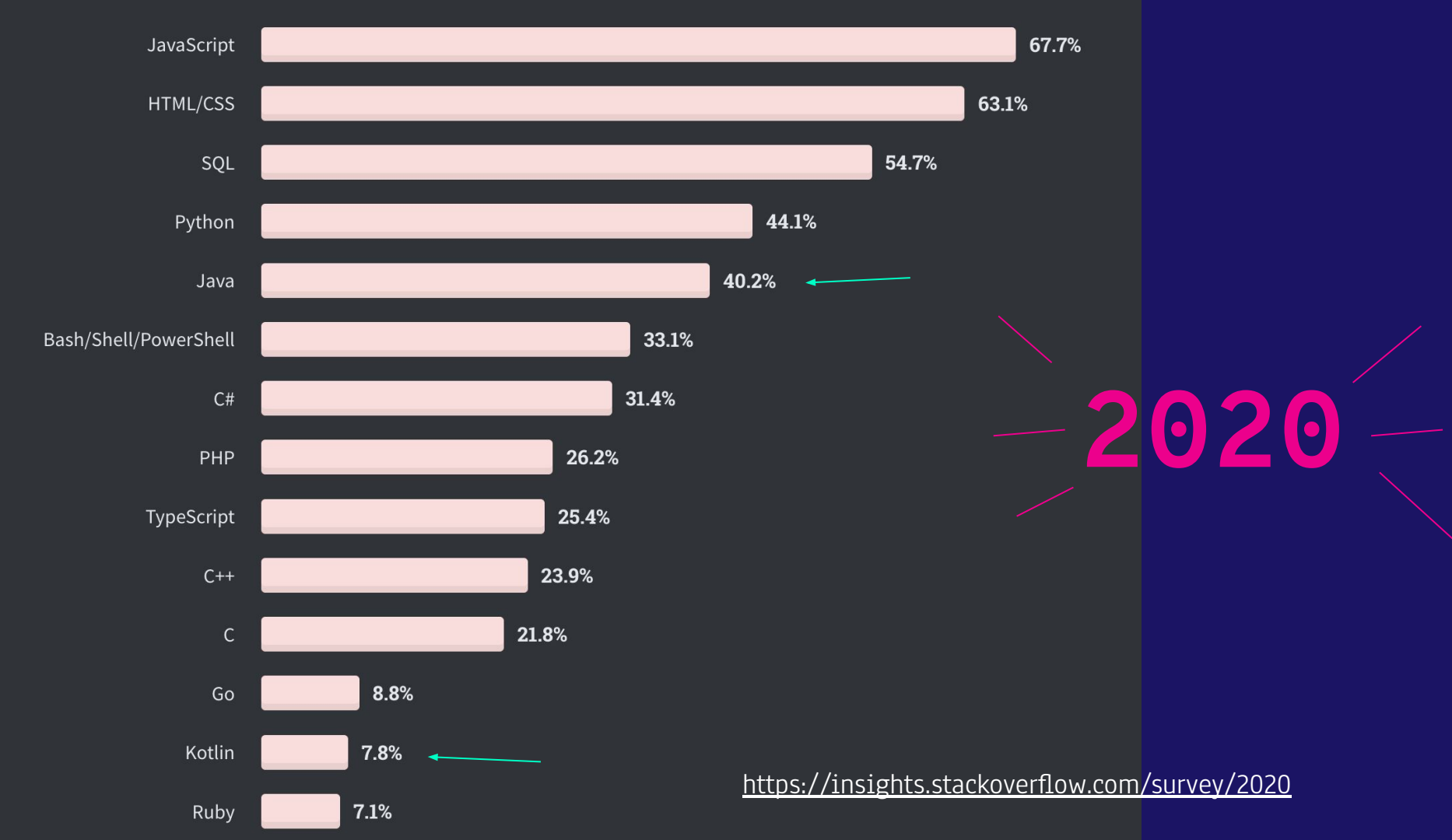
FLUX
(Example)

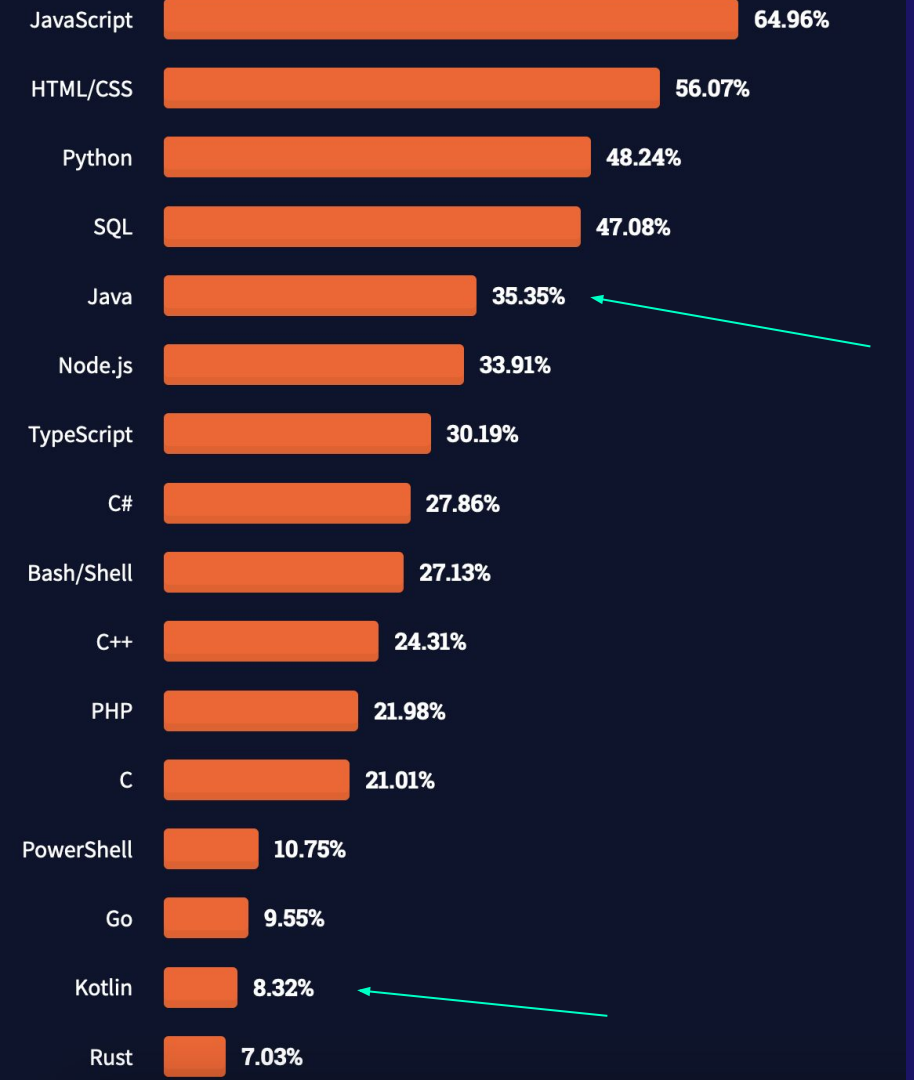
```
package com.example.demo.controller

import org.springframework.web.bind.annotation. GetMapping
import org.springframework.web.bind.annotation. RestController
import reactor.core.publisher. Mono

@RestController
class GreetingController  {

    @GetMapping
    fun sayHello(): Mono<String> {

        return Mono.just("Hello")
    }

}
```

Public is not necessary

: is return type

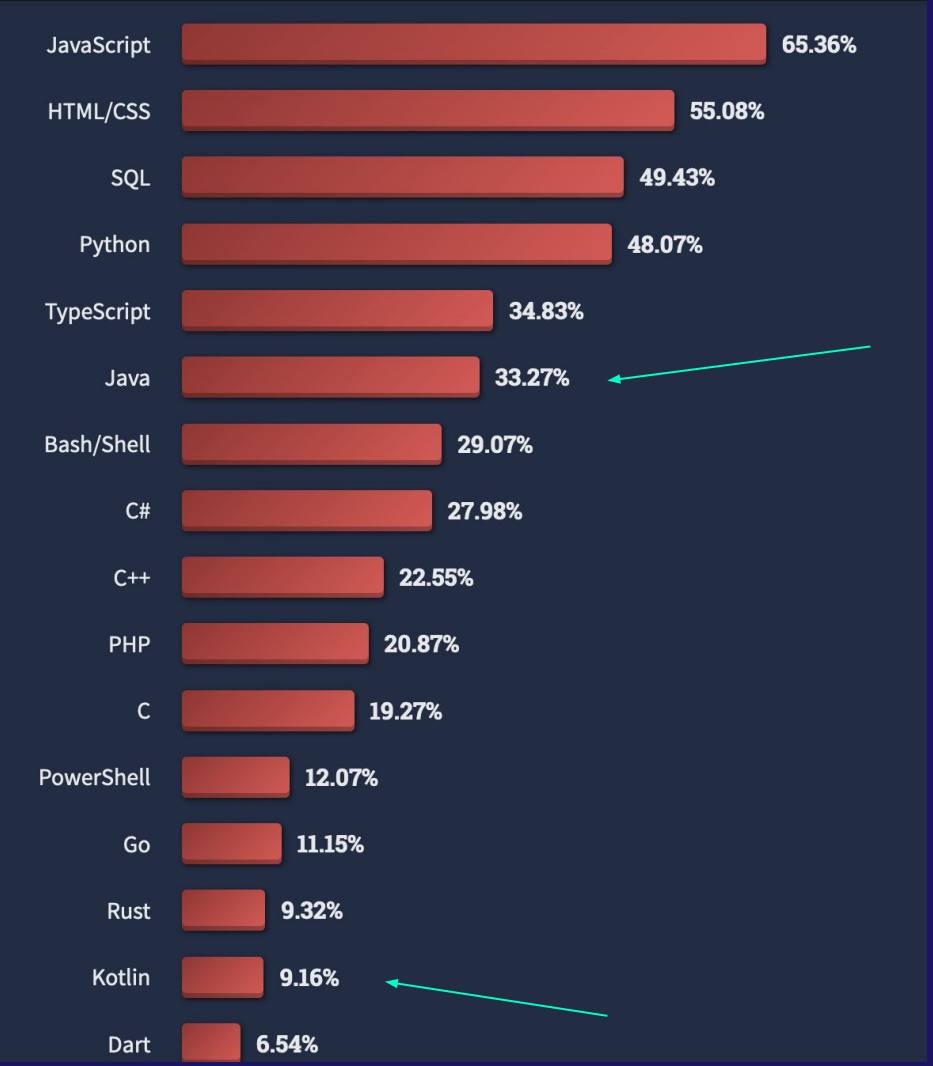
Fun = function


JavaScript
67.7%
HTML/CSS
63.1%
SQL
54.7%
Python
44.1%
Java
40.2%
Bash/Shell/PowerShell
33.1%
C#
31.4%
PHP
26.2%
TypeScript
25.4%
C++
23.9%
C
21.8%
Go
8.8%
Kotlin
7.8%
Ruby
7.1%
2020
https://insights.stackoverflow.com/survey/2020



https://insights.stackoverflow.com/survey/2021

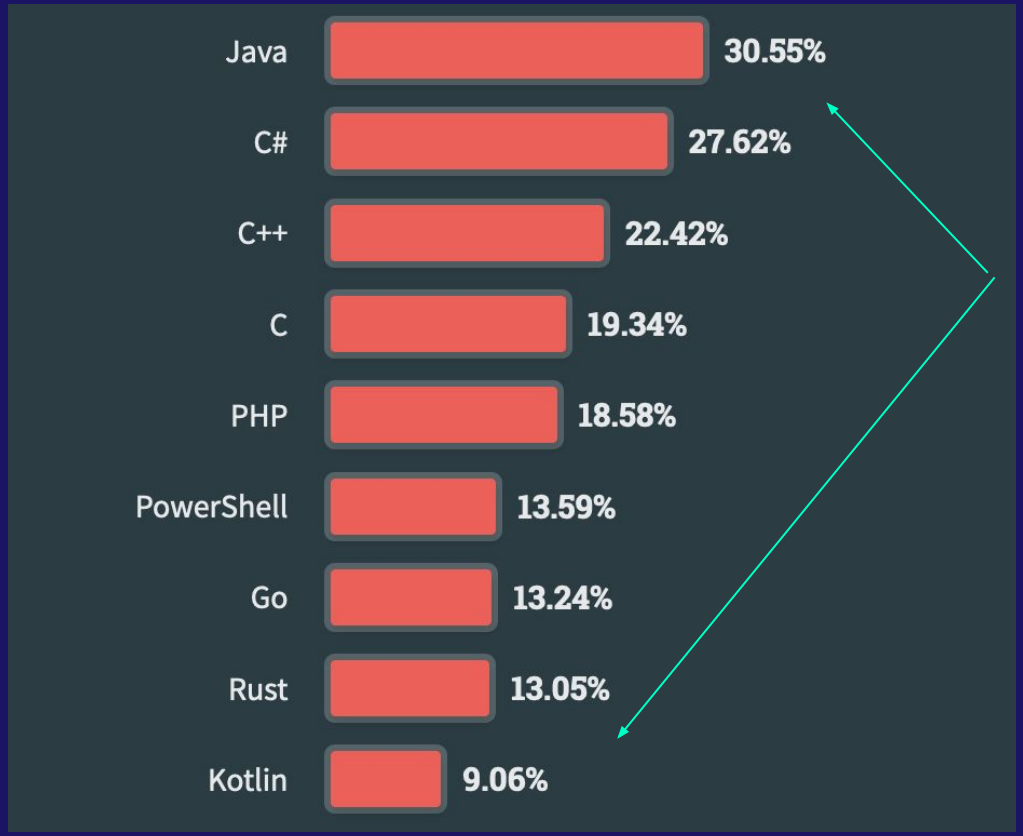
2021



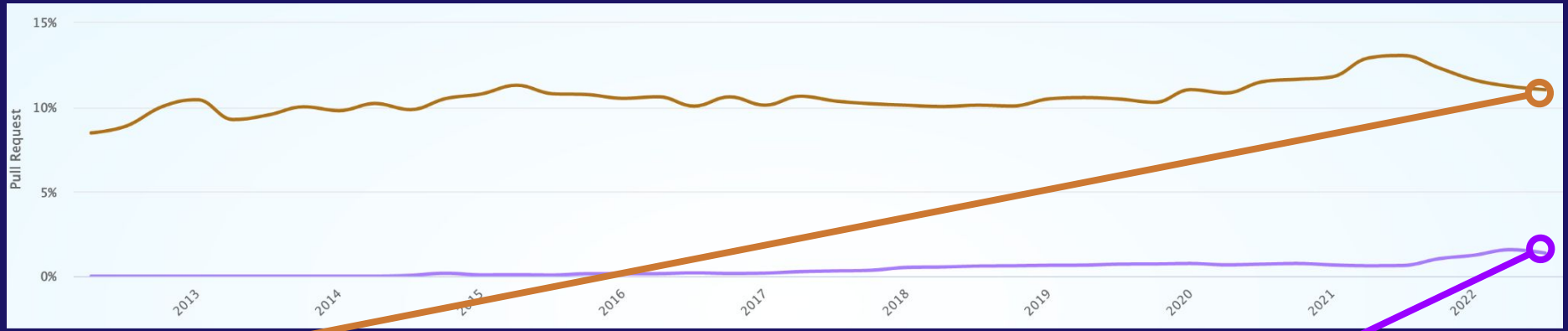
https://survey.stackoverflow.co/2022/

2022

https://survey.stackoverflow.co/2023/



2023

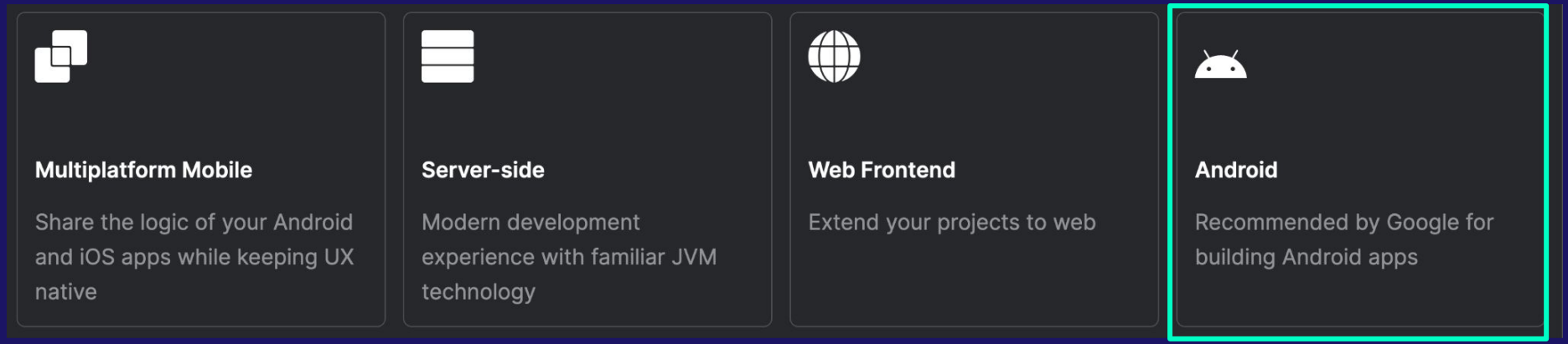


Antal 'Pull Requests' 2022

Out of all languages on projects at GitHub
https://madnight.github.io/githut/#/pull_requests/2022/3


Why Kotlin?

Why Kotlin?



Source: https://kotlinlang.org/

Why Kotlin?

“Modern, concise and safe programming language

Easy to pick up, so you can create powerful applications immediately.”

Source: https://kotlinlang.org/

```
data class Employee(
    val name: String,
    val email: String,
    val company: String
) // + automatically generated equals(), hashCode(), toString(), and copy()

object MyCompany {                                  // A singleton
    const val name: String = "MyCompany"
}

fun main() {                                        // Function at the top level
    val employee = Employee("Alice",                // No `new` keyword
        "alice@mycompany.com", MyCompany.name)
    println(employee)
}
```

No more GETTERS & SETTERS

In Kotlin, getters and setters are autogenerated:

```
package com.baeldung

class Person {
  var id: Long = 0
  var name: String? = null
  var brand: String? = null
  var price: Long = 0
}
```

Modification of getter/setter visibility can also be changed, but keep in mind that the getter's visibility must be the same as the property's visibility.

In Kotlin, every class comes with the following methods (can be overridden):

- *toString* (readable string representation for an object)
- *hashCode* (provides a unique identifier for an object)
- *equals* (used to compare two objects from the same class to see if they are the same)

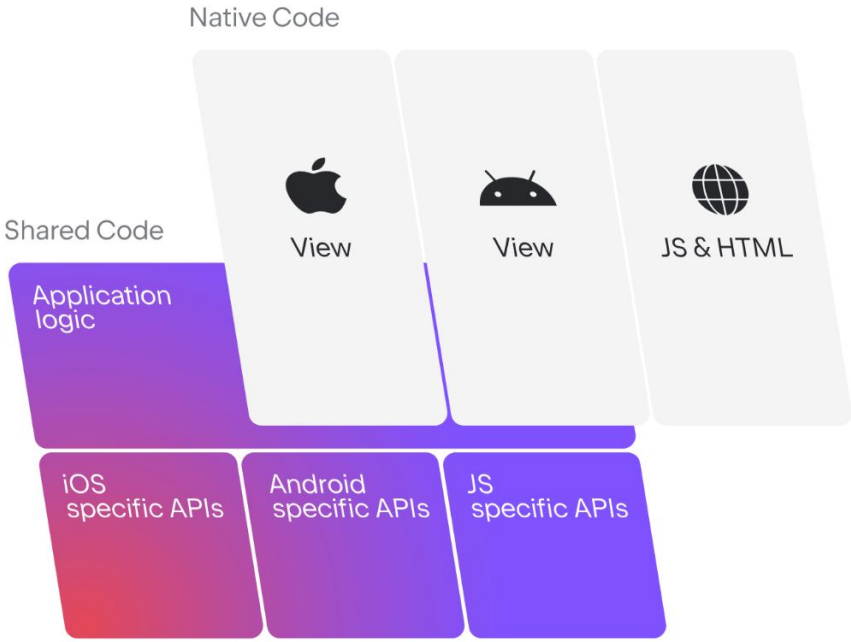
Why Kotlin?

Cross-platform layer for native applications

Share application logic between web, mobile, and desktop platforms while keeping an experience native to users.

Save time and get the benefit of unlimited access to features specific to these platforms.

Learn about Kotlin Multiplatform →



Why Kotlin?

Ramverket Spring har även stöd för 'Kotlin' !


Kotlin Syntax

Print()

Debugging

Mindre syntax
Inga semikolon

Denna funktion loggar i konsolen “Hello, World!”
detta bör inte vara något som är förvånansvärt, men vi börjar enkelt!

```kotlin
// KOTLIN SYNTAX
println("Hello, World!")



// JAVA SYNTAX
System.out.println("Hello, World!");
```

Variabler

Int, String

Vi börjar med en versal, till skillnad från Java som kör små bokstäver!

Arrayer

Här kan vi specificera datatypen för en array med IntArray, BooleanArray etc..

Variabler

Vi definierar variabler med nyckelorden: var & val

Datatyper

Varje gång vi ska definiera en datatyp för variabler, så skriver vi:

var ‘name’ : ‘datatype’ = value

Variables

```kotlin
// Kotlin
val a: Int
val b = 21

var c: Int
var d = 25

d = 23
c = 21
```

```java
// Java
final int a;
final int b = 21;

int c;
int d = 25;

d = 23;
c = 21;
```

Inom Kotlin, skriver vi datatypen på höger sida av namn deklaration. Märk av att vi har **kolon** som symboliserar datatypen.

Val = final / const
Var = Generell variable

För att tilldela ett nytt värde skriver vi bara:
namnet på variabeln = nytt värde

Strings

```kotlin
// Kotlin
val name = "John"
val lastName = "Smith"
val result = "My name is: $name $lastName"
val otherText = "My name is: ${name.substring(2)}"

val text = """
        First Line
        Second Line
        Third Line
          """
print("$name $lastName $result $otherText $text")


// Java
String name = "John";
String lastName = "Smith";
String text = "My name is: " + name + " " +
lastName;
String otherText = "My name is: " +
name.substring(2);

String text = "First Line\n" + "Second Line\n"
+ "Third Line";
```

Concatenate AKA sammanfogning av strängar, är otroligt mycket lättare inom Kotlin, och gör utveckling mycket snabbare!

Märk av att vi också kan kalla på metoder inom strängar!
${ } ← Indikerar metod anrop!

name.substring() ← indikerar startpunkt på strängen

John blir då istället → ‘hn’
Vi börjar på index 2 trotts allt!

Loops

Vi kan skriva for loops inom en rad.

Vi kan såklart också lägga till en body { }

```
// Kotlin
for (item in collection) println(item) For Each

for (i in 10 downTo 0) println(i) // Counts down
for (i in 0..10) println(i)    // Counts up
for (i in 10 downTo 0 step 2)      // Every 2:nd
```

Arrays

```kotlin
// Kotlin
val myArray: IntArray = arrayOf(0, 2, 5)
val myArrayList: ArrayList<Int> = ArrayList(
     arrayListOf(0, 15, 25)
     )

for (item in myArray) {
   println(item)
}

for (item in myArrayList) {
   println(item)
}
```

När vi tittar på ‘arrays’ så ser vi att det finns alternativ för ‘list’ och såklart traditionella arrayer!

Man brukar prata om .indices()
När det kommer till loops + arrayer.

Vi kommer se exempel på detta senare…

Arrays

Mutable och Immutable finns även som listor och kan vara enklare att komma ihåg

```
// Mutable list with explicit type declaration
val shapes: MutableList<String> = mutableListOf(
  "triangle", "square", "circle"
  )

println(shapes) // [triangle, square, circle]
```

when()

```
// KOTLIN SYNTAX

val x = 5 // some value

val xResult = when (x) {
   0, 11         -> "0 or 11"
   in 1..10      -> "from 1 to 10"
   !in 12..14    -> "not from 12 to 14"

   else -> if (isOdd(x)) {
          "is odd"
     } else {
          "otherwise"
     }
}


// When with Variables?
```

‘When’ AKA java’s Switch:
Ett strukturerat alternativ mot flertal if-satser!

when()#2

```kotlin
// KOTLIN SYNTAX

val colors = setOf("Red", "Green", "Blue")

for (color in colors) {
    when(color) {
        "Red" -> break
        "Green" -> continue
        "Blue" -> println("This is blue")
    }
}
```

Vi kan också välja att styra flödet i t.ex en for loop!

‘Break’ avbryter processen,
‘Continue’ fortsätter processen

```kotlin
// KOTLIN SYNTAX
fun example() {
}

fun calculate(): Int {

    return 2+2
}

fun calculateSubtract(x: Int, y: Int): Int {

    return x - y
}
```

fun()

Metoder skrivs med nyckelordet ‘fun’

Returtyp specificeras efter parametrar AKA ()

Parametrar och argumentets datatyp, specificeras efter variabel namn följt av kolon!

x : String

04

Uppgifter & Eget Arbete

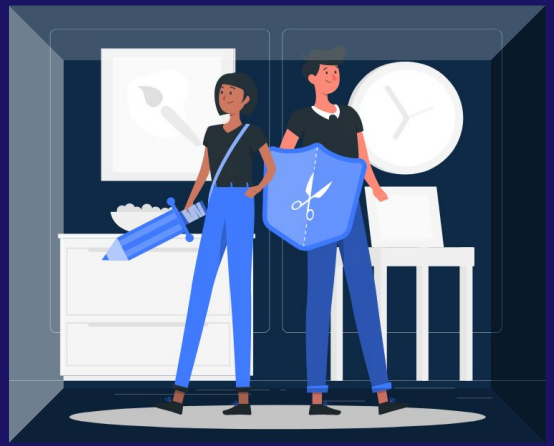
Uppgifter

Välkommen till Uppgifterna!

Uppgifterna är till för att testa dina färdigheter och kunskaper för att både öva och repetera på det vi har arbetat med under föreläsningarna.

Dessa är **INTE** obligatoriska.

Men är **starkt** rekommenderat att arbeta med!



```
1   // -Uppgift #1- //
2
3   /* INSTRUCTIONS
4
5       Skapa ett nytt projekt med Spring Init
6       WebService_Uppgifter_Lektion_3
7
8       Prova med 'Gradle'
9       Lägg till 'Dependency Spring Web'
10
11      Skapa sedan två klasser:
12      'Counter'
13      'CounterController'
14
15  */
16
17  // HINT & Examples
18  hint(" https://start.spring.io/  ")
19
20
21
22
23
```

Getting started is all about doing the 'basics' first...

```
1   // -Uppgift #2- //
2
3   /* INSTRUCTIONS
4
5       Inom 'Counter' class:
6       Initialisera två privata variabler.
7
8       Datatyper ska vara av typen 'int'
9
10      Namnge dessa till:
11          id
12          value
13
14      Avsluta genom att generera Constructor +
15      getters!
16  */
17
18  // HINT & Examples
19  hint(" Setting up the class is important,
20  since we will be using it to display the
21  information further down the line with the
22  "GetMapping" annotation! ")
23
```

Remember that @RestController marks the class as a controller but for 'web services'

```
1   // -Uppgift #3- //
2
3   /* INSTRUCTIONS
4
5       Inom 'CounterController' klassen:
6       gör detta till en kontroller för
7       web services.
8
9       Skapa en @GetMapping annotation
10      Returnera nu din 'Counter'
11
12  */
13
14  // HINT & Examples
15  hint(" Syftet är att öva på att skapa
16  'controllers' och enkla klasser som sedan
17  returneras inom 'mappings' ")
18
19
20
21
22
23
```

After you're done with this step, you can launch the application and go to /counter!

Don't forget to input localhost:8080

Välkommen till Uppgifterna!

BONUS UPPGIFTER FÖR KOTLIN

```
1   // –Uppgift #1– //
2
3   /* INSTRUCTIONS
4
5       Skapa ett nytt projekt ‘kotlin-practice’
6
7   */
8
9   // HINT & Examples
10  hint("
11  https://start.spring.io/ ← välj Kotlin
12   ")
13
14
15
16
17
18
19
20
21
22
23
```

Skapa projekt!

```
1   // –Uppgift #2– //
2
3   /* INSTRUCTIONS
4
5       Skapa en variabel för följande datatyper:
6         +   Int
7         +   String
8         +   Double
9         +   Long
10        +   Boolean
11
12      Gör nu om dessa till en ‘konstant’.
13
14      Glöm inte att tilldela variablerna
15      ett värde!
16
17  */
18
19  // HINT & Examples
20  hint(”
21  Prova gärna att plussa variablerna med varandra
22  inom en Println() så får du övat på debugging!
23   ”)
```

Övning ger färdighet, börja enkelt med variabler!

```
1           // –Uppgift #3– //
2
3   /* INSTRUCTIONS
4
5        Skapa en vanlig Array
6        Skapa en ArrayList
7
8        Tilldela initiala värden av valfri datatyp.
9        Antalet element bestämmer du själv.
10
11       När ni är klara, skriv ut sista elementet.
12
13  */
14
15  // HINT & Examples
16  hint(" Slide #55 går igenom Arrays.
17
18  För att komma åt sista elementet behöver vi komma
19  åt storleken på arrayen på något vis.
20
21   ")
22
23
```

Arrays är en datastruktur, storlek kan variera. Detta är viktigt när vi försöker komma åt 'sista' värdet i datastrukturen.

```
1           // –Uppgift #4– //
2
3   /* INSTRUCTIONS
4
5        Skapa en for loop för varje punkt, som kan:
6          +   Räkna upp från 0 till 25
7          +   Räkna ner från 20 till 5
8          +   Räknar upp via vartannat värde till 50
9          +   Skriver ut varje element inom ‘array’
10
11  */
12
13  // HINT & Examples
14  hint(”
15  Slide #54 går igenom Loops.
16  Slide #55 går igenom Arrays.
17
18  För att komma åt sista elementet behöver vi komma
19  åt storleken på arrayen på något vis.
20   ”)
21
22
23
```

For loops - är ett sätt att upprepa sig själv utan att behöva skriva om kod multipla gånger.
Använd dig av denna metod för att lösa problem som kräver upprepning!

Minska din börda

```
1   // –Uppgift #5– //
2
3   /* INSTRUCTIONS
4
5        Med en vanlig array,
6        sätt en ‘initial’ storlek på arrayen.
7        Därefter, sätt alla dessa värden med en int.
8
9   */
10
11  // HINT & Examples
12  hint(" Detta har vi ej gått igenom!
13  Leta efter svaret här: kotlin docs ")
14
15
16
17
18
19
20
21
22
23
```

Likt inom Java, så går det att sätta ett initialt värde på en array vilket sätter storleken på arrayen!

```
1   // –Uppgift #6– //
2
3   /* INSTRUCTIONS
4
5       Försök skapa en RestController med fyra
6   endpoints:
7
8       “/add”,
9       “/subtract”,
10      “/multiply”,
11      “/divide”
12
13      Ta gärna hjälp av ‘@PathVariables’!
14
15  */
16
17  // HINT & Examples
18  hint(”
    https://www.baeldung.com/spring-pathvariable
     ”)
19
20
21
22
23
```

MINNS DU?

```
// Varför döper vi klasserna:
// Greeting
// GreetingController

// Hur kommer det sig att de har samma
// 'greeting' namn? De är väl olika?
```

MINNS DU?

```
// Varför skriver man /ex?name=value

// Vad är ?
// Vad är name
// vad är =
// Vad är 'value'
```

RÄTT ELLER FEL

1. **@RestController och @Controller** annotation gör samma sak

2. **@RestController** tillkommer med **@ResponseBody** annotation

3. **@GetMapping** är ett sätt att definiera 'endpoints'

4. **Asynchronous**, gör kallelser till 'non-blocking'

Markera texten med muspekaren för att se svar!

Namnge Status Koder

- 200 -
- 201 -

- 302 -

- 400 -
- 401 -
- 403 -
- 404 -
- 408 -
- 405 -
- 415 -
- 418 -

- 500 -

Bonus: Kan du förklara status kodernas innebörd?

Markera texten med muspekaren för att se svar!

Svar: på nästa sida!

SVAR

- 200 - OK
- 201 - Created

- 302 - Found

- 400 - Bad Request
- 401 - Unauthorized
- 403 - Forbidden
- 404 - Not Found
- 408 - Request Timeout
- 405 - Method Not Allowed
- 415 - Unsupported Media Type
- 418 - I'm a teapot

- 500 - Internal Server Error

Bonus: Kan du förklara status kodernas innebörd?

Markera texten med muspekaren för att se svar!

THANKS !

Do you have any questions?
kristoffer.johansson@sti.se

sti.learning.nu/

You can also contact me VIA Teams (quicker response)
Du kan också kontakta mig VIA Teams (Snabbare svar)